UNIVERSIDADE REGIONAL DE BLUMENAU

CENTRO DE CIÊNCIAS EXATAS E NATURAIS

CURSO DE CIÊNCIA DA COMPUTAÇÃO – BACHARELADO

IAR 2.0: API OTIMIZADA PARA APLICAÇÕES DE ENTRETENIMENTO EM REALIDADE AUMENTADA NA PLATAFORMA IOS

ÁLLAN MILBRATZ DE CAMARGO

BLUMENAU
2013

2013/1-05

ÁLLAN MILBRATZ DE CAMARGO

IAR 2.0: API OTIMIZADA PARA APLICAÇÕES DE ENTRETENIMENTO EM REALIDADE AUMENTADA NA PLATAFORMA IOS

Trabalho de Conclusão de Curso submetido à Universidade Regional de Blumenau para a obtenção dos créditos na disciplina Trabalho de Conclusão de Curso II do curso de Ciência da Computação — Bacharelado.

Prof. Dalton Solano dos Reis, M. Sc. - Orientador

BLUMENAU
2013

2013/1-05

IAR 2.0: API OTIMIZADA PARA APLICAÇÕES DE ENTRETENIMENTO EM REALIDADE AUMENTADA NA PLATAFORMA IOS

Por

ÁLLAN MILBRATZ DE CAMARGO

Trabalho aprovado para obtenção dos créditos na disciplina de Trabalho de Conclusão de Curso II, pela banca examinadora formada por:

Presidente:	Prof. Dalton Solano dos Reis, M. Sc. – Orientador, FURB
Membro:	Prof. Aurélio Faustino Hoppe, M. Sc. – FURB
Membro:	Prof. Antonio Carlos Tavares, M. Sc. – FURB

Blumenau, 9 de julho de 2013

Dedico este trabalho a comunidade acadêmica, na esperança de que seja tão instrutivo para o leitor quanto foi para mim desenvolvê-lo.

AGRADECIMENTOS

A meu pai, por ter me apresentado ao mundo da informática e tecnologia.

À minha mãe, por me ensinar o valor de dedicação e excelência.

À minha irmã, pelos empurrões e cobranças.

Aos meus amigos, pelos bons momentos de diversão e trabalho.

Ao meu orientador, Dalton Solano dos Reis, pelas instruções na área gráfica e por ter acreditado na conclusão deste trabalho.

Limites não existem. O que existe são platôs, mas você não deve permanecer neles, deve ir além.

Bruce Lee

RESUMO

Neste trabalho é apresentada uma metodologia de codificação *assembler* com instruções SIMD para dispositivos ARM. Com foco em otimizar uma aplicação de realidade aumentada, este trabalho descreve os passos utilizados para se atingir um ganho global de 57% em performance. São descritos também os ganhos em nível de algoritmo, onde atingiu-se um tempo de execução 31 vezes melhor que a versão anterior.

Palavras-chave: iOS. Dispositivos móveis. Realidade aumentada. Assembler. ARM. SIMD.

ABSTRACT

This paper presents a methodology for coding *assembler* with SIMD instructions for ARM devices. With its focus in optimizing an augmented reality application, this paper describes the steps taken to achieve a global gain of 57% in performance. There are, also, descriptions for the gains at an algorithm level, where the final version achieved an execution time 31 times faster than the last version.

Key-words: iOS. Mobile devices. Augmented reality. Assembler. ARM. SIMD.

LISTA DE ILUSTRAÇÕES

Figura 1 – Representação gráfica de uma instrução SIMD 16
Figura 2 – Quadro da aplicação executando 17
Figura 3 – AR Defender executando em um iPhone 4 18
Figura 4 – Aplicação demonstrando a funcionalidade da API Vuforia 19
Figura 5 – Diagrama de casos de uso 21
Quadro 1 – Caso de uso UC06 em detalhes 21
Figura 6 – Diagrama de classes 23
Figura 7 – Diagrama de sequência 25
Quadro 2 – Método `shouldAutorotate` 27
Figura 8 – Itens necessários da biblioteca OpenCV 28
Figura 9 – Estrutura do projeto representada pela IDE 29
Figura 10 – Protótipo da interface no ambiente Xcode 29
Quadro 3 – Exemplo de trecho de código onde foi utilizada a classe `StopWatch` 30
Quadro 4 – Método `neonAdaptiveThreshold` 31
Quadro 5 – Implementação de *box filter* 33
Quadro 6 – Implementação de *box filter* com *intrinsics* 34
Quadro 7 – Implementação de *box filter* em *assembler* 35
Figura 11 – Estrutura do projeto `iAR2` 37
Figura 12 – Configuração do *target* `arUCO` 38
Figura 13 – Configuração do *target* `iAR2` 38
Figura 14 – Lista de bibliotecas utilizadas na compilação 38
Figura 15 – Interface do iOS e ícone da aplicação 39
Figura 16 – Aplicação de testes executando 39
Quadro 8 – Comparação entre código C++ e *assembler* 41
Quadro 9 – Consumo de memória 41
Quadro 10 – Desempenho global da aplicação 42
Quadro 11 – Desempenho de cada etapa de RA 42
Quadro 12 – Desempenho das três versões do algoritmo de *threshold* 43
Quadro 13 – Comparação entre trabalhos correlatos 44
Quadro 14 – Versão original do loop de *threshold* 50
Quadro 15 – Versão otimizada com *intrinsics* 51

Quadro 16 – Execução da instrução `cle` 51
Quadro 17 – Versão codificada em *assembler* 52
Quadro 18 – Caso de uso UC01 em detalhes 53
Quadro 19 – Caso de uso UC02 em detalhes 53
Quadro 20 – Caso de uso UC03 em detalhes 54
Quadro 21 – Caso de uso UC04 em detalhes 54
Quadro 22 – Caso de uso UC05 em detalhes 55
Quadro 23 – Descrição da classe `iARAppDelegate` 56
Quadro 24 – Descrição da classe `iARViewController` 56
Quadro 25 – Descrição da classe `VideoInput` 57
Quadro 26 – Descrição da classe `GLView` 57
Quadro 27 – Descrição da classe `Render` 57
Quadro 28 – Descrição da classe `Object` 58
Quadro 29 – Descrição da classe `ObjectManager` 58
Quadro 30 – Descrição das classes `RuntimeInfo` e `Flags` 58
Quadro 31 – Descrição das classes `ParserProtocol` e `WavefrontParser` 58
Figura 17 – Diagrama de sequência 59

LISTA DE SIGLAS

API – *Application Programming Interface*

ASM – *Assembly Language*

ARMv7 – *Advanced RISC Machine version 7*

ArUco – *Augmented Reality University of Cordoba*

FPS – *Frames Per Second*

GLSL – *OpenGL Shading Language*

iAR – *iPhone Augmented Reality*

IDE – *Integrated Development Environment*

iOS – *iPhone Operating System*

MacOS – *Macintosh Operating System*

OpenCV – *Open source Computer Vision*

OpenGL – *Open Graphics Library*

OpenGL ES – *OpenGL for Embedded Systems*

RA – *Realidade Aumentada*

RISC – *Reduced Instruction Set Computing*

SDK – *Software Development Kit*

SIMD – *Single Instruction Multiple Data*

SSE – *Streaming SIMD Extensions*

UML – *Unified Modeling Language*

SUMÁRIO

1 INTRODUÇÃO 13
1.1 OBJETIVOS DO TRABALHO 14
1.2 ESTRUTURA DO TRABALHO 14
2 FUNDAMENTAÇÃO TEÓRICA 15
2.1 PROGRAMAÇÃO PARALELA EM DISPOSITIVOS MÓVEIS 15
2.1.1 Instruções SIMD 15
2.1.2 Unidade Processadora de Gráficos 16
2.2 "EXPLORANDO MODELOS VIRTUAIS 3D COM REALIDADE AUMENTADA NO SDK DO IPHONE" 17
2.3 TRABALHOS CORRELATOS 18
2.3.1 AR Defender 18
2.3.2 Vuforia 19
3 DESENVOLVIMENTO 20
3.1 REQUISITOS PRINCIPAIS DO PROBLEMA A SER TRABALHADO 20
3.2 ESPECIFICAÇÃO 20
3.2.1 Diagrama de casos de uso 20
3.2.1.1 Visualizar detalhes de desempenho 21
3.2.2 Diagrama de classes 24
3.2.2.1 Classe `MarkerDetector` 24
3.2.2.2 Classe `Marker` 24
3.2.2.3 Classe `StopWatch` 24
3.2.2.4 Cabeçalhos `arucoARMUtilz.h` e `imgproc.hpp` 24
3.2.3 Diagrama de sequência 25
3.3 IMPLEMENTAÇÃO 26
3.3.1 Preparação do ambiente de desenvolvimento 26
3.3.2 Atualização do projeto e bibliotecas 27
3.3.2.1 OpenCV 27
3.3.2.2 ArUco 28
3.3.3 Nova interface para monitorar o desempenho 29
3.3.4 Codificação das rotinas otimizadas 30
3.3.4.1 Otimização do *box filtering* 32

3.3.5 Operacionalidade da implementação 36
3.3.5.1 Estrutura do projeto e compilação 36
3.3.5.2 Aplicação de testes 38
3.4 RESULTADOS E DISCUSSÃO 40
3.4.1 Consumo de memória 41
3.4.2 Desempenho 42
3.4.3 Características 43
4 CONCLUSÕES 45
4.1 EXTENSÕES 46
REFERÊNCIAS BIBLIOGRÁFICAS 47
APÊNDICE A – Detalhamento do método `neonAdaptiveThreshold` 50
ANEXO A – Casos de uso da aplicação de testes 53
ANEXO B – Descrições das classes 56
ANEXO C – Diagrama de sequência 59

1 INTRODUÇÃO

Os dispositivos móveis, especificamente *smartphones*, vem assumindo cada vez mais funções no cotidiano. No princípio serviam apenas como telefones e plataformas de entretenimento. Logo viu-se a evolução destes dispositivos com a adição de recursos como câmera, comunicação de rede sem fio, sensores de movimento e sistema de posicionamento global, além de processadores mais rápidos e unidades processadoras de gráficos, do inglês *Graphics Processing Unit* (GPU). Com tantos recursos, principalmente de processamento, tornou-se comum o desenvolvimento de aplicações para estes dispositivos, aplicações que antes eram apenas viáveis em computadores de mesa (*Desktop*). Apesar de dispositivos como o iPhone (APPLE INC, 2011) possuírem tecnologias como GPU e mais de um núcleo de processamento, poucas aplicações utilizam estes recursos eficientemente.

Aplicações de Realidade Aumentada (RA) são um exemplo onde o uso eficiente dos recursos é essencial para o sucesso da aplicação, visto que o volume de dados a ser processado é grande e os resultados devem ser obtidos em tempo real (AZUMA, 1997, p. 357). Além das características citadas, "a RA é um complemento natural a computação móvel, já que um sistema móvel de RA pode direcionar o usuário diretamente em diversas situações. [...]" (WAGNER; SCHMALSTIEG, 2003, p. 1).

Segundo Wagner e Schmalstieg (2003, p. 5), terceirizar o processamento para um servidor, por exemplo, pode resolver a limitação de processamento do dispositivo móvel. Criando tarefas paralelas nestas aplicações, obtém-se um aproveitamento melhor dos recursos a disposição. No cenário atual, para se implementar este paralelismo, não é mais necessário delegar as tarefas a um servidor. Com tecnologias como o processador com dois ou mais núcleos de processamento e a GPU, pode-se criar o paralelismo dentro do próprio dispositivo, sem dependências com comunicações de rede, por exemplo.

Diante deste cenário, adaptou-se a *Application Programming Interface* (API) desenvolvida por Hess (2011a) para utilizar programação paralela. Segundo Hess (2011a), a principal característica a ser melhorada na API, chamada de *iPhone Augmented Reality* (iAR), é o desempenho. Para este fim estudou-se: formas de implementação de processamento paralelo na plataforma móvel iOS, processamento paralelo de imagens e o funcionamento do mecanismo de reconhecimento de marcadores, utilizado na RA.

1.1 OBJETIVOS DO TRABALHO

O objetivo deste trabalho é a criação de rotinas utilizadas em RA para serem executadas de forma paralela. As rotinas criadas são integradas a API desenvolvida por Hess (2011a), consolidando a extensão do trabalho e gerando uma nova iteração da API.

Os objetivos específicos do trabalho são:

a) melhorar o desempenho das rotinas de reconhecimento utilizando mais de um núcleo de processamento e instruções do tipo *Single Instruction Multiple Data* (SIMD);
b) otimizar o uso da GPU nas tarefas gráficas e verificar a possibilidade de se utilizar este processador para outras tarefas da API.

1.2 ESTRUTURA DO TRABALHO

Este trabalho é composto de quatro capítulos. O próximo capítulo contém a fundamentação teórica necessária para a compreensão dos temas abordados na implementação.

O terceiro capítulo contempla a descrição do desenvolvimento deste trabalho, onde são descritos os casos de uso e os diagramas de classe e sequência. No terceiro capítulo também são discutidos os resultados obtidos com a implementação do projeto.

Por último, o quarto capítulo expõe as considerações finais e conclusões, finalizando com sugestões de extensões para projetos futuros.

2 FUNDAMENTAÇÃO TEÓRICA

A seção 2.1 possui informações sobre programação paralela em dispositivos móveis e suas aplicações. Em seguida, a seção 2.2 descreve brevemente o trabalho a ser estendido pelo presente e na seção 2.3 são apresentados dois trabalhos correlatos ao exposto.

2.1 PROGRAMAÇÃO PARALELA EM DISPOSITIVOS MÓVEIS

Com a evolução dos *smartphones* surgiram novas técnicas para se implementar paralelismo. Estas técnicas envolvem o uso de recursos como as instruções SIMD e unidades processadoras de gráficos. A seguir estes recursos serão detalhados individualmente.

2.1.1 Instruções SIMD

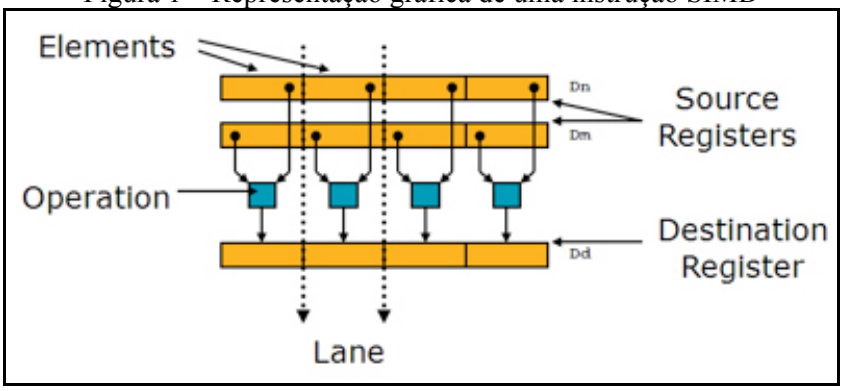
A necessidade de otimizar o uso de registradores foi a motivação inicial para criar instruções que trabalham com múltiplos dados (LEUPERS, 2000). Segundo Leupers (2000), a maioria dos processadores trabalha com palavras de 32 bits, porém, aplicações de áudio e vídeo necessitam de apenas 16 ou 8 bits de precisão, resultando em possível desperdício de recursos. Portanto, processadores modernos possuem instruções SIMD que além de agilizar o processamento dos dados, também utilizam de forma eficiente os registradores do processador. As principais aplicações das instruções SIMD são em processadores de vídeo, imagem e áudio, onde o volume de dados, na maioria dos casos, é grande.

Segundo a taxonomia de Flynn (1972), instruções SIMD são consideradas paralelismo, pois possibilitam uma mesma operação sobre vários valores simultaneamente. Exemplos de implementações SIMD são: o conjunto de instruções NEON, criado por Arm Ltd (2012), e o conjunto *Streaming SIMD Extensions* (SSE), criado por Intel Corporation (2008). A implementação NEON está presente em todas as versões da linha Cortex-A (ARM LTD, 2012), usada constantemente em dispositivos móveis. A implementação SSE é encontrada a partir dos processadores Intel Pentium III (INTEL CORPORATION, 2008) até os mais recentes como o Intel i7, estes utilizados em desktops e laptops.

A implementação NEON possibilita executar uma operação SIMD sobre até oito valores inteiros de 16 ou 32 bits simultaneamente (ARM LTD, 2012). Além de valores inteiros, também são suportados valores em ponto flutuante de 32 e 64 bits. Segundo Arm Ltd (2009), o processador Cortex-A9 possui registradores de entrada e saída dedicados as instruções SIMD. Vale lembrar que as instruções SIMD são executados por um módulo a parte dentro do processador ARM, portanto, é necessário copiar os valores para os registradores do referido módulo. Os registradores funcionam como vetores, onde até oito

valores do mesmo formato são armazenados (ARM LTD, 2012). Uma vez inicializados com os valores, a instrução SIMD pode ser executada sobre os registradores de entrada como mostra a Figura 1. A Figura 1 ressalta também a existência de vários valores dentro de cada registrador SIMD, estas linhas de valores são chamadas de *lanes* e cada instrução executada utiliza todas as *lanes* dos registradores envolvidos.

Figura 1 – Representação gráfica de uma instrução SIMD



Fonte: Arm Ltd (2012).

2.1.2 Unidade Processadora de Gráficos

Uma das tarefas mais custosas da RA, em termos de desempenho, é a renderização do espaço tridimensional sobre os quadros da câmera. Segundo Boggan e Pressel (2007, p. 1), renderizar gráficos é uma tarefa matematicamente intensiva que, se for assumida somente pela *Central Processing Unit* (CPU), pode causar perda de desempenho. Tendo em vista este cenário, Boggan e Pressel (2007, p. 1) afirmam que "Para trazer este trabalho da CPU para um hardware que foi otimizado para esta tarefa, computadores pessoais, estações de trabalho e consoles de jogos usam uma GPU. [...]". Em dispositivos móveis modernos como o iPhone 4S (APPLE INC, 2011), encontra-se o uso de GPUs para acelerar a execução de todas as tarefas gráficas, inclusive de cenas tridimensionais em jogos. Para explorar os recursos da GPU será necessário utilizar a biblioteca *Open Graphics Library* (OpenGL) em sua versão para dispositivos móveis, designada *Embedded Systems* (ES) (KHRONOS GROUP, 2007).

Além de tarefas gráficas, Boggan e Pressel (2007, p. 1) sugerem que a GPU seja utilizada para paralelizar ou até terceirizar outras tarefas como processamento de imagens e cálculos matemáticos extensos. Em dispositivos móveis esta técnica ainda não foi vastamente explorada, porém, existem algumas soluções comerciais em desenvolvimento como a biblioteca *Jacket Native Development Kit* (ACCELEREYES, 2011).

2.2 "EXPLORANDO MODELOS VIRTUAIS 3D COM REALIDADE AUMENTADA NO SDK DO IPHONE"

Em 2011 Hess provou que é possível executar um sistema de RA em um dispositivo móvel, neste caso o *smartphone* iPhone 4. A ferramenta, chamada de "iAR", foi desenvolvida em formato de API, onde as funções facilitam a criação de um aplicação de RA. Segundo Hess (2011a, p. 63), a API desenvolvida possui as seguintes características:

a) posicionamento de objetos a partir da identificação de marcadores;
b) suporta até 1024 marcadores diferentes;
c) interação em tempo real;
d) desenvolvida para a plataforma iOS.

A Figura 2 ilustra um quadro da aplicação resultante sendo executada. Pode-se observar na parte superior da imagem o desempenho da aplicação sendo mostrado no formato *Frames Per Second* (FPS).

Figura 2 – Quadro da aplicação executando

Fonte: Hess (2011a, p. 57).

Segundo Hess (2011a, p. 60), apesar de alguns algoritmos terem alcançado desempenho individual satisfatório, a aplicação ainda não executa dentro de um padrão produtivo. De acordo com os testes realizados, a aplicação atinge em média 1,5 FPS, sendo que o ideal para uma aplicação seria no mínimo 20 FPS. Hess aponta que a tarefa mais custosa, que consequentemente causa a perda de desempenho, é a identificação dos marcadores. Considerando a identificação dos marcadores como a principal fonte de lentidão, Hess sugere como extensão de seu trabalho a melhora do desempenho da API. Para tanto é sugerido ainda a reescrita dos algoritmos de tratamento de imagens utilizando instruções SIMD. Por último,

Hess sugere também melhorias visuais, como a texturização dos objetos e melhorias nas interfaces expostas ao desenvolvedor.

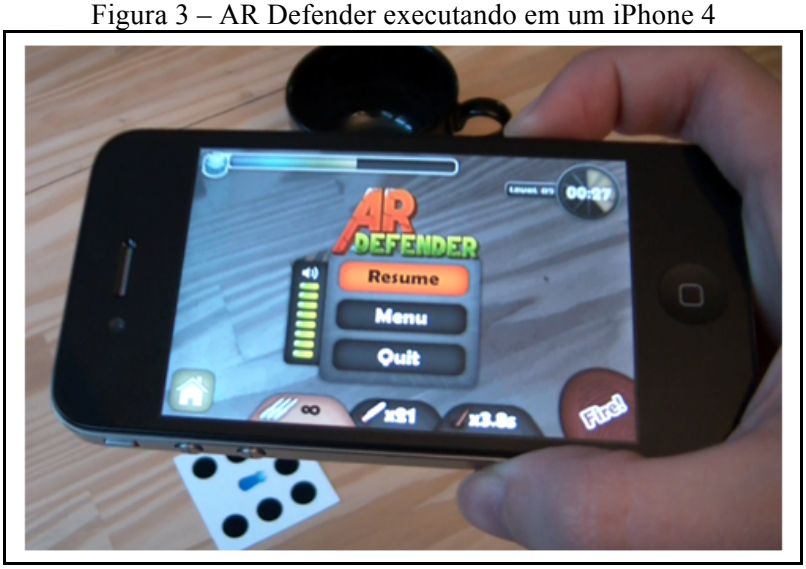
2.3 TRABALHOS CORRELATOS

Trabalhos dedicados a processamento paralelo em dispositivos móveis foram encontrados, porém foram poucos e estes tinham o acesso restritos a usuários dos seus respectivos serviços de publicação. Portanto, optou-se por citar projetos com bom desempenho e estabilidade, estes projetos são o aplicativo AR Defender (INT13, 2010) e a API Vuforia (QUALCOMM, 2012a).

2.3.1 AR Defender

O AR Defender foi desenvolvido pela INT13, uma produtora de jogos para dispositivos móveis. Este é um jogo do gênero *tower defense*, onde o jogador vê na tela do dispositivo uma torre sendo atacada por diversos inimigos, como helicópteros e soldados. O objetivo é proteger a torre dos agressores, atirando neles com a mira presente na tela do dispositivo. O AR Defender contempla todas as características de um sistema de RA, como o registro dos objetos 3D no meio físico e a execução em tempo real (INT13, 2010). A aplicação está disponível para as plataformas iOS, Google Android e BadaOS (INT13, 2010). A Figura 3 ilustra o jogo executando em um iPhone 4. Nota-se o marcador, utilizado como referência pela aplicação, atrás do dispositivo.

Figura 3 – AR Defender executando em um iPhone 4



Fonte: Int13 (2010).

2.3.2 Vuforia

A API Vuforia é desenvolvida e mantida pela Qualcomm, fabricante de *chipsets* e processadores para diversos tipos de dispositivos. A Vuforia é um conjunto de funções e recursos onde o desenvolvedor pode criar facilmente aplicações de RA em dispositivos móveis (QUALCOMM INC, 2012a). A API está disponível em forma de biblioteca para as plataformas iOS e Google Android e para a plataforma Unity 3D está disponível em forma de biblioteca externa (QUALCOMM INC, 2012a). A Vuforia é disponibilizada gratuitamente para desenvolvedores de todo o mundo e, além disso, é concedido também o direito de utilizar a API em aplicações comerciais.

Uma das peculiaridades da Vuforia está no reconhecimento de marcadores. A API é capaz de reconhecer texturas como sendo marcadores. A Figura 4 mostra uma aplicação feita com a Vuforia reconhecendo uma textura dentro da área impressa de uma folha de papel.

Figura 4 – Aplicação demonstrando a funcionalidade da API Vuforia

Fonte: Qualcomm Inc (2012b).

3 DESENVOLVIMENTO

Este capítulo trata de assuntos pertinentes a implementação do projeto. Aqui encontram-se requisitos, especificação, detalhes da implementação e instruções para a utilização da biblioteca. Encerrando o capítulo, encontram-se os resultados e discussões.

3.1 REQUISITOS PRINCIPAIS DO PROBLEMA A SER TRABALHADO

Os requisitos que identificam a API são:

a) permitir a captura de imagens através da câmera digital do dispositivo móvel (Requisito Funcional - RF);
b) utilizar processamento paralelo em tarefas complexas e/ou extensas, tais como: conversão de imagens para escalas de cinza e detecção de bordas (RF);
c) disponibilizar de forma paramétrica os algoritmos de RA necessários para se criar uma aplicação de entretenimento (RF);
d) ser implementada para a plataforma iOS (Requisito Não Funcional - RNF).

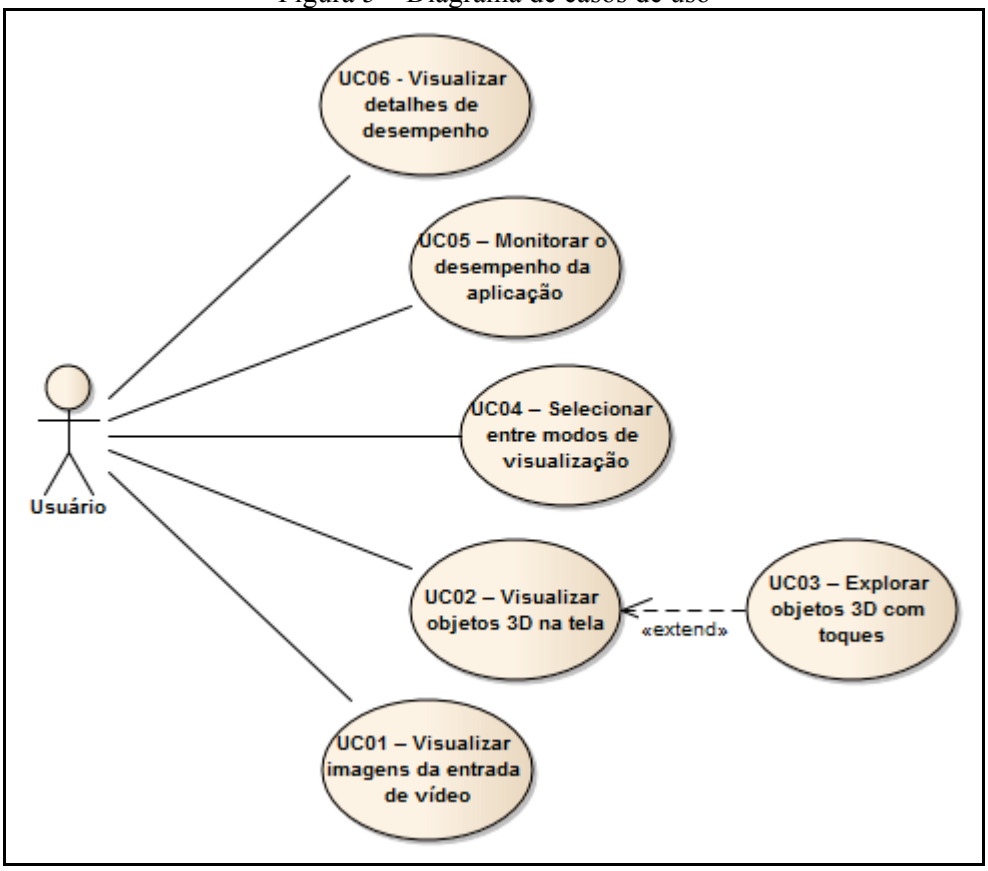
3.2 ESPECIFICAÇÃO

Para especificar a implementação do projeto, foram utilizados diagramas da *Unified Modeling Language* (UML), sendo eles os diagramas de casos de uso, classes e sequência. A ferramenta Enterprise Architect 9.3 foi utilizada para modelar e gerar os diagramas.

3.2.1 Diagrama de casos de uso

Nesta seção serão vistos os casos de uso para a aplicação de testes. Todos os casos de uso da aplicação desenvolvida por Hess, em 2011, foram mantidos. Foi acrescentado o caso de uso UC06, o qual contempla a extensão da interface de monitoramento. Vale lembrar que, segundo Hess (2011a, p. 31), o ator conceitual nomeado `Usuário` também representa a figura do desenvolvedor, principalmente porque o objetivo da aplicação de testes é validar o desempenho das rotinas desenvolvidas. Detalhes sobre os casos de uso UC01 a UC05 podem ser encontrados no Anexo A.

Figura 5 – Diagrama de casos de uso



Fonte: adaptado de Hess (2011a, p. 31).

3.2.1.1 Visualizar detalhes de desempenho

O caso de uso UC06, descrito em detalhes no Quadro 1, representa a interação entre o `Usuário` e a funcionalidade que permite a visualização dos tempos de cada etapa do processo de RA.

Quadro 1 – Caso de uso UC06 em detalhes

UC06 – Visualizar detalhes de desempenho	
Descrição	Permite ao `Usuário` visualizar o tempo consumido por cada etapa do processo de RA, em tempo de execução.
Pré-Condição	O processo de RA deve estar executando e a monitoração de desempenho, descrita no caso de uso UC05, deve estar ativa.
Cenário principal	1. O `Usuário` carrega a aplicação de RA; 2. O motor de RA executa todas as rotinas padrão; 3. O `Usuário` toca no botão de detalhes de tempo; 4. A aplicação inverte o valor da *flag* que armazena o status do detalhamento de desempenho; 5. A interface mostra o painel com informações de tempo para cada etapa do processo de RA.
Exceção	Na etapa 5, as informações podem não ser mostradas caso o processo de RA ainda não tenha sido executado.
Pós-condição	O `Usuário` passa a ser capaz de visualizar, em tempo de execução, o tempo que cada etapa do processo de RA está consumindo.

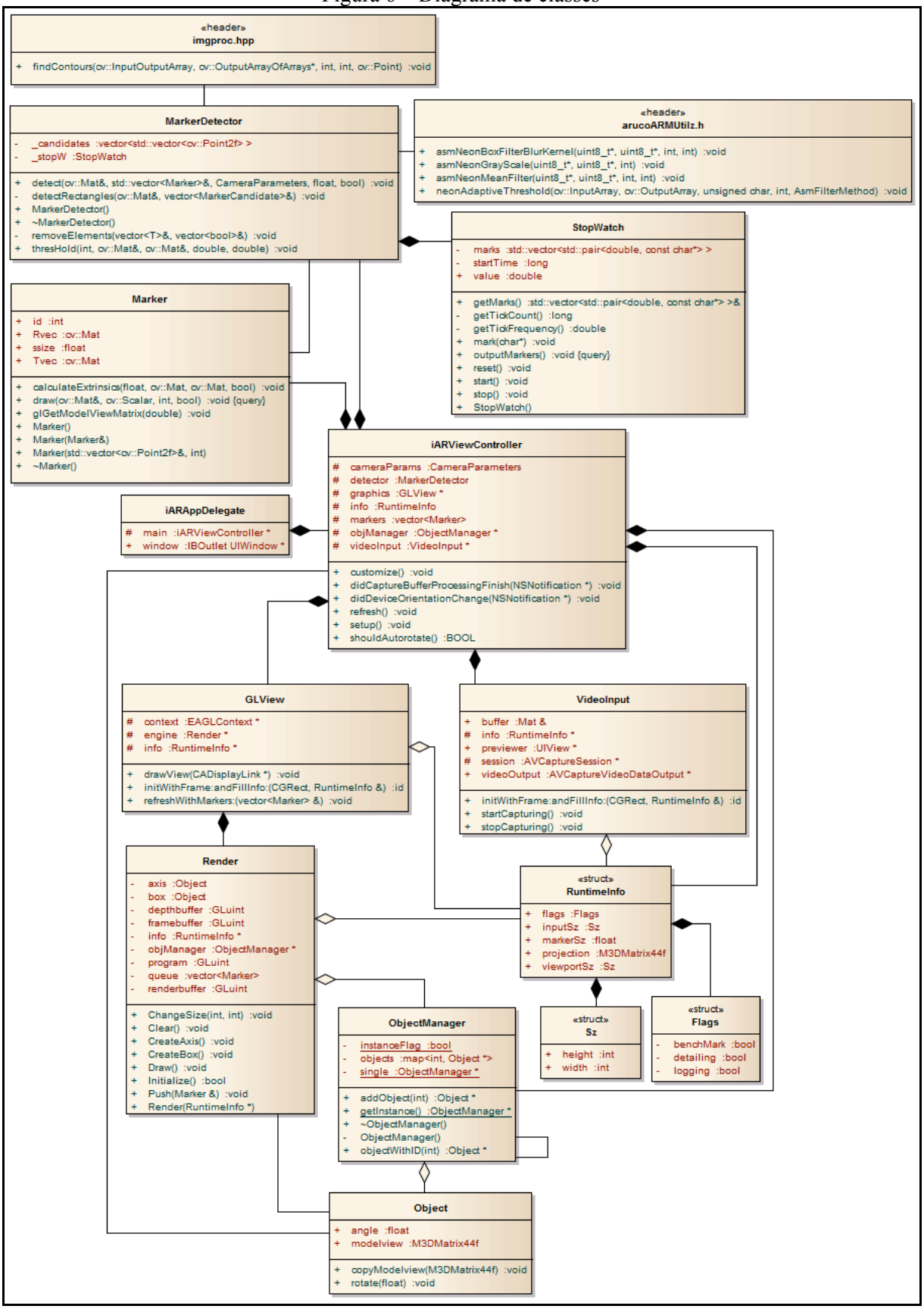
3.2.2 Diagrama de classes

Especificações das classes e suas relações nesta implementação, estão definidas no diagrama de classes ilustrado na Figura 6, o qual é uma versão atualizada do diagrama criado por Hess (2011a, p. 35). A principal modificação no diagrama foi a adição de cinco novos elementos, sendo eles:

a) as classes `MarkerDetector` e `Marker`, que já existiam na versão original e fazem parte da biblioteca ArUco (SALIÑAS, 2012);
b) a classe `StopWatch`, criada para armazenar o tempo de cada etapa do processo de RA;
c) os cabeçalhos `arucoARMUtilz.h` e `imgproc.hpp`.

Após a Figura 6, os novos elementos são descritos em maior detalhe. Os elementos que foram mantidos do diagrama original têm sua descrição detalhada no Anexo B.

Figura 6 – Diagrama de classes



Fonte: adaptado de Hess (2011a, p. 35).

3.2.2.1 Classe `MarkerDetector`

A classe `MarkerDetector` é o coração do motor de RA, onde todas as chamadas específicas do processo são realizadas. A *pipeline* de RA é iniciada a partir do método `detect`, o qual é chamado pela classe `iARViewController` a cada novo quadro recebido da câmera. Uma peculiaridade da classe é que ela não armazena os marcadores detectados, eles são encaminhados pela lista passada como parâmetro na chamada do método `detect`.

Chamadas externas dos cabeçalhos `arucoARMUtilz.h` e `imgproc.hpp` são feitas para cumprir tarefas especializadas, sendo elas os métodos `asmNeonGrayScale`, `neonAdaptiveThreshold` e `findContours`.

Por ser o centro do processo de RA, a classe `MarkerDetector` também é responsável por registrar o tempo de cada etapa através da classe utilitária `StopWatch`.

3.2.2.2 Classe `Marker`

A classe `Marker` armazena informações referentes a um marcador encontrado: seu identificador, seu tamanho e as coordenadas na imagem onde foi detectado. A principal responsabilidade da classe `Marker` é fornecer a matriz de posicionamento para os modelos 3D que serão renderizados na tela. A classe é utilizada pelas classes `MarkerDetector` e `iARViewController` para armazenar o resultado da execução do processo de RA e para calcular as matrizes de posicionamento, respectivamente.

3.2.2.3 Classe `StopWatch`

A classe `StopWatch` é responsável por armazenar marcas de tempo, estas marcas são compostas de um nome e o tempo onde ela foi feita. A classe funciona de forma similar a um cronometro esportivo onde se inicia a contagem e em seguida são marcadas as voltas. O método `start` inicia a contagem enquanto o método `mark` registra uma nova marca.

3.2.2.4 Cabeçalhos `arucoARMUtilz.h` e `imgproc.hpp`

O cabeçalho `arucoARMUtilz.h` contém rotinas otimizadas para processadores ARM utilizando linguagem *assembler* e instruções SIMD. As rotinas presentes neste cabeçalho fazem o processamento das imagens recebidas da câmera. Neste trabalho, o cabeçalho `arucoARMUtilz.h` foi adicionado a biblioteca ArUco para ajudar na organização do código e também na compilação.

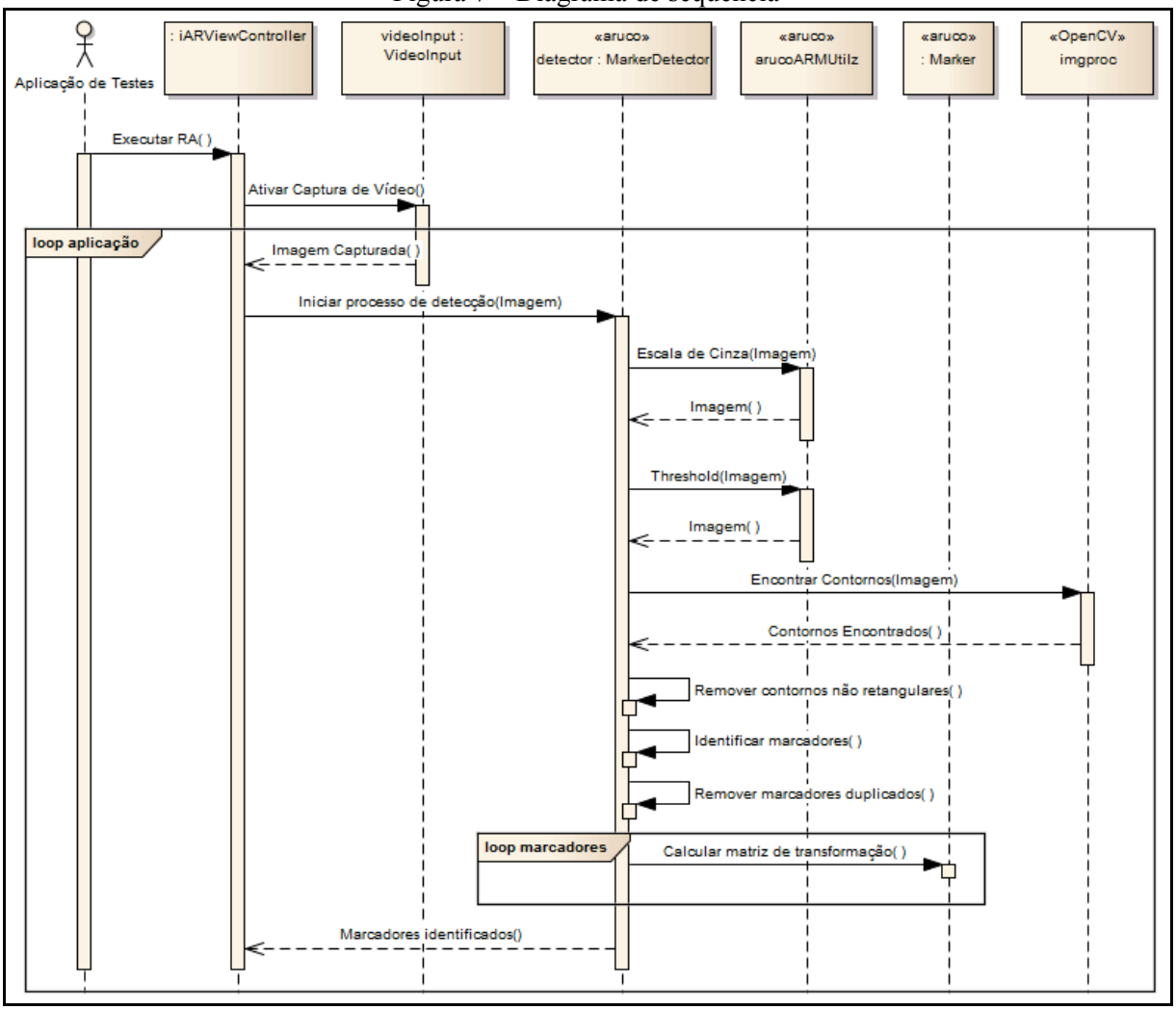
O cabeçalho `imgproc.hpp` faz parte da biblioteca OpenCV (OPENCV WIKI, 2013) e é de suma importância na *pipeline* de RA pois contém o método responsável por detectar

contornos na imagem. O método `findContours` busca por contornos na imagem de entrada e retorna uma lista com os resultados.

3.2.3 Diagrama de sequência

Para ilustrar o funcionamento da *pipeline* de RA, são apresentadas as sequências de chamadas desde o recebimento de uma imagem da câmera até o retorno dos marcadores identificados. O diagrama de sequência da Figura 7 descreve especificamente as etapas da *pipeline* de RA, abstraindo os demais procedimentos executados pela aplicação. O diagrama de sequência global, ilustrando toda a aplicação, pode ser encontrado no Anexo C.

Figura 7 – Diagrama de sequência



O fluxo de RA inicia com uma nova imagem recebida da câmera. Ao receber esta imagem da instância `videoInput`, a classe `iARViewController` inicia o processo de RA fazendo a chamada da função `detect` de `MarkerDetector`.

Recebida a imagem, a classe `MarkerDetector` realiza a chamada de `asmNeonGrayScale` do cabeçalho `arucoARMUtilz.h` convertendo a imagem em escalas de

cinza. O último passo de preparação da imagem é a chamada da função `neonAdaptiveThreshold` do cabeçalho `arucoARMUtilz.h`. Esta função é responsável por ressaltar bordas presentes nas imagens utilizando um filtro e então transformar o resultado deste filtro em uma imagem preto e branco, baseando-se na luminosidade detectada na imagem.

Em seguida, é feita a chamada da função `findContours` de `imgproc.hpp`, esta irá fornecer uma lista de contornos encontrados na imagem. Estes contornos são submetidos a um processo de seleção, onde serão mantidos somente aqueles com formato retangular. Os contornos selecionados são identificados para que sejam agora chamados de marcadores. Após a identificação são filtrados os marcadores repetidos.

Antes de encerrar o fluxo de detecção, é necessário calcular as matrizes de posicionamento de cada marcador através da função `calculateExtrinsics` da classe `Marker`. Uma vez encerrado o fluxo de detecção, a aplicação segue com as tarefas de desenho na tela.

3.3 IMPLEMENTAÇÃO

Nesta seção são apresentadas ferramentas e técnicas utilizadas na implementação das rotinas otimizadas e a operacionalidade da API.

3.3.1 Preparação do ambiente de desenvolvimento

Para o desenvolvimento na plataforma iOS utiliza-se o *Integrated Development Environment* (IDE) Xcode em conjunto com o *Software Development Kit* (SDK). O ambiente Xcode está disponível somente para a plataforma Macintosh *Operating System* (MacOS). Utilizou-se a versão 10.8.3 do sistema MacOS, a versão 4.6.2 do Xcode e a versão 6.1 do SDK para iOS. O dispositivo utilizado no desenvolvimento foi o *smartphone* iPhone 4S executando a versão 6.1.3 do sistema operacional iOS.

No desenvolvimento utilizou-se as linguagens C++, Objective-C++ e *assembler* para processadores ARM. A linguagem chamada de Objective-C++ é na verdade uma integração entre código Objective-C e C++, onde o primeiro consegue realizar chamadas do segundo, facilitando a utilização de bibliotecas.

Dando início ao desenvolvimento, foi feito o *download* do projeto original (HESS, 2011b). Em seguida foi criado um novo repositório (CAMARGO, 2013) hospedado pelo serviço BitBucket. Para registrar mudanças na codificação e manter o repositório atualizado, utilizou-se a ferramenta SourceTree na versão 1.5.6.

3.3.2 Atualização do projeto e bibliotecas

Quando importado para a nova versão da IDE o projeto não apresentou problemas de compilação, porém ao executar no dispositivo, a tela da aplicação era rotacionada de forma a impossibilitar a utilização. Investigando o problema, descobriu-se que já existia uma rotina para lidar com a rotação do dispositivo, porém o novo sistema operacional tenta rotacionar a interface de forma independente. Para evitar que esta tentativa ocorra, foi necessário implementar o método `shouldAutorotate` na classe controladora da interface, `iARViewController`. A implementação do método está ilustrada no Quadro 2, nota-se que o método simplesmente informa o sistema de que não é necessário rotacionar a interface.

Quadro 2 – Método `shouldAutorotate`

```
1  - (BOOL)shouldAutorotate
2  {
3      return NO;
4  }
```

Outra atualização importante foi quanto a compilação. O projeto original estava configurado para as arquiteturas Armv6 e Armv7. Como o smartphone iPhone 4S utiliza a arquitetura Armv7s, foi necessário adicioná-la a lista de compilação. Também optou-se por remover a arquitetura Armv6, pois os dispositivos que a implementam possuem recursos de hardware limitados e já não são mais suportados pelo iOS 6 (APPLE INC, 2012).

Nas próximas seções serão detalhados os procedimentos de atualização e compilação das bibliotecas utilizadas neste projeto.

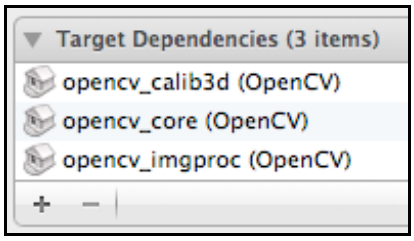
3.3.2.1 OpenCV

A biblioteca OpenCV aparece como um conjunto de utilitários por todo o projeto, principalmente na detecção de marcadores, onde tem um papel importantíssimo na detecção de contornos.

Ao utilizar a biblioteca, Hess (2011a, p. 41) encontrou diversas dificuldades sendo que para conseguir realizar a compilação, foi necessária a execução de um script criado por terceiros. Nesta iteração da API já existia uma versão oficial compilada para a plataforma iOS prevendo que o usuário utilizaria a linguagem Objective-C. Porém, neste projeto, esta versão não foi utilizada devido a incompatibilidade do código pré-compilado do OpenCV com o código existente em C++ e Objective-C++. Esta incompatibilidade ocorreu devido a diferenças na compilação de código C e C++. Compiladores C++ geram identificadores em tempo de compilação para evitar que o desenvolvedor chame uma função passando parâmetros inválidos, processo este chamado de *name mangling* (KEFALLONITIS, 2007).

Para resolver esta incompatibilidade foi necessário compilar a biblioteca para processadores ARM e para ser utilizada com C++. O primeiro passo foi fazer o *download* da versão 2.4.4 do código fontes da biblioteca (OPENCV WIKI, 2013). Em seguida, com a ajuda da aplicação CMake (KITWARE, 2013), foi gerado o projeto para Xcode. Observou-se que a compilação geraria vários arquivos de bibliotecas estáticas, a maioria não utilizada pelo projeto. Portanto, optou-se por minimizar a compilação, removendo itens que não serão utilizados futuramente. A Figura 8 demonstra a tela da IDE onde são selecionados os itens da compilação, esta contendo os três itens utilizados no projeto.

Figura 8 – Itens necessários da biblioteca OpenCV



Para selecionar quais arquiteturas a biblioteca seria compilada, foram seguidos os mesmos critérios de preparação do projeto, descritos na seção 3.3.2.

3.3.2.2 ArUco

A biblioteca ArUco tem uma papel de suma importância neste projeto, pois ela é o centro de todo o processo de identificação dos marcadores.

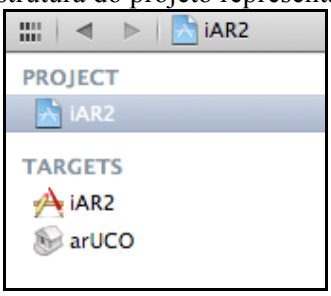
Em seu trabalho, Hess (2011b) utilizou a versão 1.0 da biblioteca. Ao consultar a página oficial da biblioteca (SALIÑAS, 2012) notou-se que estava disponível uma nova versão, 1.2.4. Fez-se então o *download* da biblioteca e a integração com o projeto. Uma das diferenças notadas foi a deprecação do método `glGetProjectionMatrix` da classe `MarkerDetector`. O método foi movido para a classe `CameraParameters` para facilitar o acesso às informações necessárias para a execução (SALIÑAS, 2012).

O método principal da classe `MarkerDetector`, `detect`, também sofreu alterações. Entretanto, nenhuma alteração modificou o processo de RA de forma significativa, portanto, o comportamento do motor permaneceu o mesmo.

Em termos de estrutura do projeto, foi alterada a forma como a biblioteca é compilada, criou-se um novo *target* (alvo) de compilação nomeado arUCO. Este novo *target* é responsável exclusivamente pela compilação da biblioteca, concedendo um nível maior de independência do resto do projeto. O resultado da compilação é a biblioteca estática `arUCO.a`, que pode ser utilizada, inclusive, em outras aplicações. A Figura 9 mostra a interface da IDE

onde são listados os *targets* de compilação, nota-se que o *target* do executável é chamado de iAR2.

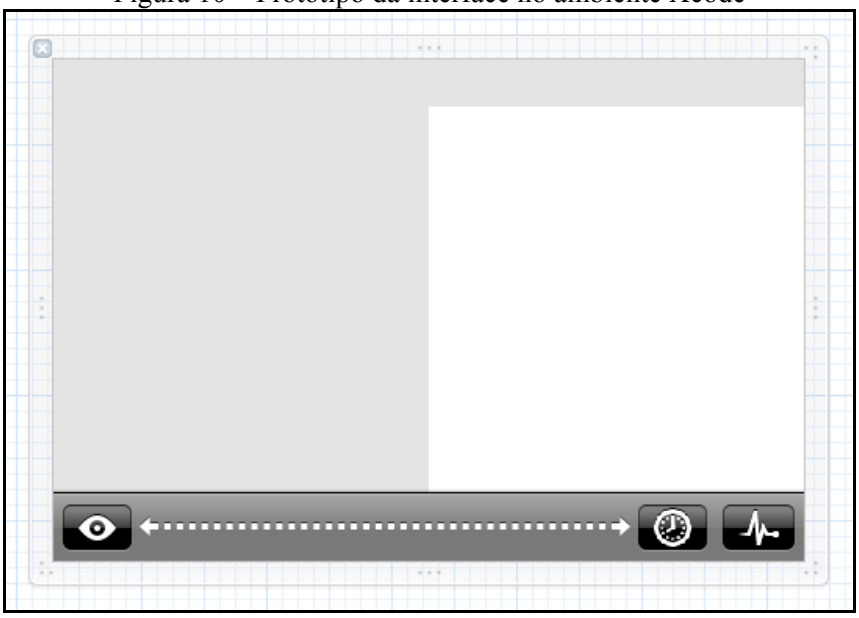
Figura 9 – Estrutura do projeto representada pela IDE



3.3.3 Nova interface para monitorar o desempenho

Para auxiliar no desenvolvimento das rotinas otimizadas, foi necessário acrescentar informações à interface existente. Diante desta necessidade, criou-se uma extensão da interface atual, onde são mostrados tempos para cada etapa do processo de RA. A Figura 10 mostra esta extensão dentro do protótipo da interface.

Figura 10 – Protótipo da interface no ambiente Xcode



A extensão é habilitada pelo botão à direita com um desenho de cronometro. Maiores detalhes sobre o comportamento da extensão podem ser vistos no caso de uso UC06, na seção 3.2.1.1.

Para fornecer os dados visualizados na interface, criou-se a classe `StopWatch`. Esta classe mantém uma lista de marcações feitas durante a execução de trechos de código. O Quadro 3 traz um trecho de código onde a classe é utilizada, neste caso, para marcar o tempo

que os procedimentos de *gray-scale* e *threshold* levaram. O trecho do Quadro 3 faz parte do método `detect` da classe `MarkerDetector`.

Quadro 3 – Exemplo de trecho de código onde foi utilizada a classe `StopWatch`

```
    ...
1   // Greyscale done with SIMD (NEON) instructions
2   asmNeonGrayScale(gryData, baseData, numPixels);
3   grey.data = gryData;
4
5   _stopW.mark("greyScale");
6
7   ///Do threshold the image and detect contours
8   thresHold (_thresMethod,grey,thres,_thresParam1,_thresParam2);
9
10  _stopW.mark("thresHold");
    ...
```

Ainda no Quadro 3, nota-se as chamadas a classe nas linhas `5` e `10`. Para mostrar os valores na tela, é chamada a função `getMarkers`. Esta função retorna a lista de marcadores registrados até o momento. Com a lista de marcadores, é feito o preenchimento do retângulo a esquerda, visto na Figura 10.

3.3.4 Codificação das rotinas otimizadas

Durante a análise preliminar de desempenho, notou-se que o procedimento de *threshold* poderia ser melhorado significativamente, pois este procedimento lida essencialmente com massa de dados.

O procedimento de *threshold* tem dois objetivos, destacar contornos presentes na imagem e transformar a imagem em preto e branco. Um filtro chamado de *box filter* (filtro de caixa) é passado na imagem, criando um efeito de borrão e assim alargando as bordas presentes na imagem. A transformação em preto e branco é feita através da diferença entre os pixels da imagem original e os da imagem filtrada. Baseando-se nessa diferença, o algoritmo designa, então, preto ou branco para cada pixel.

Na versão de Hess (2011b), a biblioteca ArUco utilizava o método `adaptiveThreshold` da biblioteca OpenCV (OPENCV WIKI, 2013). Para esta versão, foi feita uma nova implementação do método de *threshold*, baseada no original da biblioteca OpenCV. O Quadro 4 traz esta nova versão do método.

Quadro 4 – Método `neonAdaptiveThreshold`

```
1   void neonAdaptiveThreshold(const cv::InputArray _src,
    cv::OutputArray _dst, unsigned char maxValue, const int delta, const
    AsmFilterMethod filter)
2   {
3       cv::Mat src = _src.getMat();
4       CV_Assert( src.type() == CV_8UC1 );
5       cv::Size size = src.size();
6
7       _dst.create( size, src.type() );
8       cv::Mat dst = _dst.getMat();
9
10      if( maxValue < 0 )
11      {
12          dst = cv::Scalar(0);
13          return;
14      }
15
16      cv::Mat mean;
17
18      if( src.data != dst.data )
19          mean = dst;
20
21      if(filter)
22         asmNeonMeanFilter(src.data,mean.data,size.width,size.height);
23      else
24         asmNeonBoxFilterBlurKernel(src.data,mean.data,size.width,
    size.height);
25
26      uchar imaxval = cv::saturate_cast<uchar>(maxValue);
27
28      __asm__ __volatile
29      ("lsr       %[sz], %[sz], #3     \n"
30       "vdup.s8   d0, %[delta]         \n"
31       "vdup.u8   d1, %[maxval]        \n"
32       "vmov.u8   d2, #0               \n"
33       "1:                             \n"
34       "vld1.8    {d3}, [%[src]]!      \n"
35       "vld1.8    {d4}, [%[m]]!        \n"
36       "vsub.s8   d5, d3, d4           \n"
37       "vcle.s8   d6, d5, d0           \n"
38       "vbsl.u8   d6, d1, d2           \n"
39       "vst1.8    {d6}, [%[dst]]!      \n"
40       "subs      %[sz], %[sz], #1     \n"
41       "bne       1b                   \n"
42       : [dst] "+r" (dst.data)
43       : [src] "r" (src.data),[m] "r" (mean.data),[sz] "r"
    (size.width*size.height),[maxval] "r" (imaxval),[delta] "r" (-delta)
44       : "memory", "cc");
45  }
```

O método foi reimplementado respeitando a estrutura do original, com isso, os parâmetros de entrada `(1)` e o fluxo permaneceram os mesmos. No código, a imagem é filtrada utilizando o método passado pelo parâmetro `filter (21)` que é uma escolha do desenvolvedor. O método de filtro, `asmNeonBoxFilterBlurKernel (24)`, é declarado no cabeçalho `arucoARMUtilz.h` e discutido em detalhes na seção 3.3.4.1. O trecho de código

assembler (28) é responsável pela transformação em preto e branco. Este é resultado de uma otimização do loop presente na implementação original. Esta otimização não foi trivial, e as iterações necessárias para se chegar neste resultado podem ser encontradas no APÊNDICE A.

3.3.4.1 Otimização do *box filtering*

Visando maximizar o desempenho da rotina de filtro, optou-se por escrever uma nova rotina e então transformá-la em código *assembler* utilizando instruções SIMD. Segundo Merel (2011) e Pipenbrinck (2009), a melhor forma de se codificar nestas condições é seguindo três passos:

a) definir a rotina a ser convertida em *assembler* e simplificá-la ao máximo na linguagem escolhida, neste caso C/C++;
b) codificar esta rotina utilizando chamadas a *intrinsics*[1];
c) baseando-se na versão anterior, codificar em *assembler* a rotina.

Criou-se então uma implementação simplificada do *box filter* que trabalha somente com imagens em escalas de cinza, pois nesta etapa do processo de RA a imagem já foi convertida para este formato.

O processo de *box filter* é um loop onde cada pixel da imagem é avaliado, juntamente com seus vizinhos. O pixel atual no processo irá receber a média dos vizinhos, baseada no *kernel*, uma matriz com os pesos de cada vizinho. O resultado do filtro é totalmente dependente do *kernel* utilizado, portanto, foi necessário eleger uma configuração para ser utilizada nesta implementação. A matriz que gerou os melhores contornos na imagem resultante foi a que está representada junto à implementação, no Quadro 5 (linhas 9 a 11).

[1] Funções que representam instruções SIMD em alto nível (MEREL, 2011).

Quadro 5 – Implementação de *box filter*

```
1   void boxFiltering(const uint8_t* src, uint8_t* dst, const int width,
    const int height)
2   {
3       float denominator = 0.0f;
4       float grey;
5       float greySrc;
6       int iGrey;
7       int indexOffset;
8
9       const uint8_t kernel[9] = { 1,  0,  1,
10                                  0,  0,  0,
11                                  1,  0,  1};
12
13      const int indices[9] = { -(width + 1), -width, -(width - 1),
14                                      -1,       0,            1,
15                                  width - 1,  width,    width + 1};
16
17      for (int i = 0;i < 9;i++)
18      {
19          denominator += kernel[i];
20      }
21
22      for(int i = 0; i < height - 1; i++)
23      {
24          for (int j = 1;j < width - 1;j++)
25          {
26              grey = .0f;
27              indexOffset = (i * width) + j;
28
29              for (int k = 0;k < 9;k++)
30              {
31                  greySrc = src[indexOffset+indices[k]];
32                    grey += greySrc * kernel[k];
33              }
34
35              iGrey = (uint8_t)(grey / denominator);
36              dst[indexOffset] = iGrey;
37          }
38      }
39  }
```

Na linha `9` temos a inicialização da variável `kernel`, uma matriz que dita quais pixels vizinhos são considerados no cálculo da média. A média dos valores será atribuída ao pixel atual no loop. A média dos pixels é calculada de acordo com o *kernel* `(32)`, aqui é possível notar que caso um vizinho possua peso zero ele não terá influência sobre o cálculo. Na linha `35` é feita a divisão do somatório, a fim de obter a média. A matriz `indices (13)` contém valores de referência para calcular ponteiros durante o loop.

Em seguida, iniciou-se adaptações necessárias para a implementação do paralelismo das instruções SIMD. Observando o Quadro 6, nota-se que a declaração do *kernel* foi removida. Nesta versão, a seleção dos pixels vizinhos é embutida no código `(3)`. Optou-se

por sacrificar o dinamismo do *kernel*. Esta escolha reduziu significativamente a quantidade de operações necessárias para se chegar ao resultado desejado.

Quadro 6 – Implementação de *box filter* com *intrinsics*

```
1   void neonBoxFilter(uint8_t* __restrict src, uint8_t* __restrict dst,
    const int width, const int height)
2   {
3       const int indices[4] = { -(width + 1), -(width - 1), width - 1,
    width + 1 };
4
5       uint8_t* p = src + width;
6       uint8_t* d = dst + width;
7       const uint8_t* pEnd = src + (width * height);
8
9       while(p < pEnd)
10      {
11          int i = 0;
12          uint8x8_t greyAcc = vdup_n_u8(0);
13          while (i < 4)
14          {
15              uint8_t* data = p + indices[i];
16              uint8x8_t valuesN = vld1_u8(data);
17              vsra_n_u8(greyAcc, valuesN, 2);
18              ++i;
19          }
20
21          vst1_u8(d, greyAcc);
22
23          p += 8;
24          d += 8;
25      }
26  }
```

A formação do *kernel* foi bastante benéfica para a otimização do filtro. Como são considerados somente quatro vizinhos `(3)`, é possível calcular a média utilizando uma operação de *shift and accumulate* (deslocar e acumular). A instrução `sra` chamada pela função `vsra_n_u8 (17)` realiza estas operações. O loop também foi alterado, este verificando agora o ponteiro do *buffer* da imagem, eliminando a necessidade de variáveis contadoras. Além da simplificação mencionada, temos agora oito pixels sendo processados simultaneamente.

Merel (2011) e Pipenbrinck (2009) afirmam que, apesar de as chamadas *intrinsics* produzirem código relativamente rápido, deve-se usá-las somente como ferramentas de protótipo. Merel (2011) reforça seu ponto de vista explicando que o compilador não possui uma maneira de saber o que o desenvolvedor quer que aconteça com os registradores vetoriais. O que o leva a gerar código que armazena os registradores vetoriais em memória, criando acessos desnecessários a memória (Pipenbrinck, 2009). Isto pode ser verificado na própria implementação do Quadro 6, onde na linha `12` é declarada uma variável que irá armazenar oito inteiros de oito bits cada.

Portanto, para aproveitar ao máximo os recursos do processador, é necessário trazer a rotina para *assembler*. No Quadro 7 está a versão final, posicionada dentro do código C/C++ utilizando o padrão *inline assembler* (montagem em linha).

Quadro 7 – Implementação de *box filter* em *assembler*

```
1   void asmNeonBoxFilterBlurKernel(const uint8_t* __restrict src,
    uint8_t* __restrict dst, const int width, const int height)
2   {
3       const int indices[4] = { -(width + 1), -(width - 1), width - 1,
    width + 1 };
4
5       __asm__ __volatile__
6       ("mul           r4, %[w], %[h]        \n"
7        "lsr           r4, r4, #3            \n"
8        "add           %[src], %[src], %[w]  \n"
9        "add           %[dst], %[dst], %[w]  \n"
10       "1:                                  \n"
11       "mov           r5, #4                \n"
12       "mov           r6, %[i]              \n"
13       "vmov.u8       d0, #0                \n"
14       "2:                                  \n"
15       "ldr           r7, [r6]              \n"
16       "add           r8, %[src], r7        \n"
17       "vld1.8        {d1}, [r8]            \n"
18       "vsra.u8       d0, d1, #2            \n"
19       "add           r6, #4                \n"
20       "subs          r5, r5, #1            \n"
21       "bne           2b                    \n"
22       "vst1.8        {d0}, [%[dst]]!       \n"
23       "add           %[src], #8            \n"
24       "subs          r4, r4, #1            \n"
25       "bne           1b                    \n"
26       : [dst] "+r" (dst)
27       : [src] "r" (src), [i] "r" (indices), [w] "r" (width), [h] "r"
    (height)
28       : "memory", "cc", "r4", "r5", "r6", "r7", "r8");
29  }
```

Nota-se que este método trouxe da última versão somente os parâmetros de entrada `(1)` e a declaração dos índices de referência `(3)`. Na linha `5` está a declaração do bloco *assembler*, o identificador `__asm__`. Na mesma linha, o identificador `__volatile__` informa para o compilador que este trecho deve ficar fora de otimizações automáticas, isto elimina a possibilidade de o compilador alterar o código (KIPP, 2012). Outro aspecto importante do bloco são as declarações de registradores de entrada `(27)` e saída `(26)`. Estes registradores podem ter identificadores atribuídos a eles, por exemplo, na declaração do parâmetro `src` `(27)` onde ele recebe o identificador `[src]`. Identificadores devem sempre estar entre colchetes e serem acessados utilizando o símbolo de porcentagem `(8)`. Na linha `28` temos um componente chamado de *clobber list*, esta lista contém os componentes do processador que serão utilizados no bloco (KIPP, 2012). São selecionados os itens memória, registrador de

controle `cc` e registradores multiuso de `r4` a `r8`. Como os registradores vetoriais fazem parte de um módulo à parte, este sendo o NEON, não é necessário informá-los na lista.

Feitas as preparações iniciais, é possível fazer o detalhamento do algoritmo. O código começa com a inicialização do contador `r4` e dos ponteiros `[src]` e `[dst]`, nas linhas `6` a `9`. Em seguida são preparadas as variáveis utilizadas no loop interno, o qual aplica o *kernel* aos oito pixels processados por iteração. A instrução `vld1.8` carrega oito inteiros de 8 bits para o registrador vetorial `d1 (17)`, baseando-se no ponteiro armazenado em `r8 (16)`. A instrução `vsra.u8` realiza as operações necessárias para o cálculo da média `(18)`, armazenando o resultado no acumulador `d0`. Em seguida, a instrução `subs` faz o decremento do contador `r5` e atualiza os registradores de comparação do processador `(20)`. A linha `21` continua o loop caso a última comparação não tenha sido igual a zero. No término do loop interno, a instrução `vst1.8` transfere para a memória os oito valores processados `(22)`. Na mesma linha, é feito o incremento do ponteiro `[dst]` através do símbolo `!`, economizando uma instrução de soma. O método encerra com o final do bloco *assembler*.

3.3.5 Operacionalidade da implementação

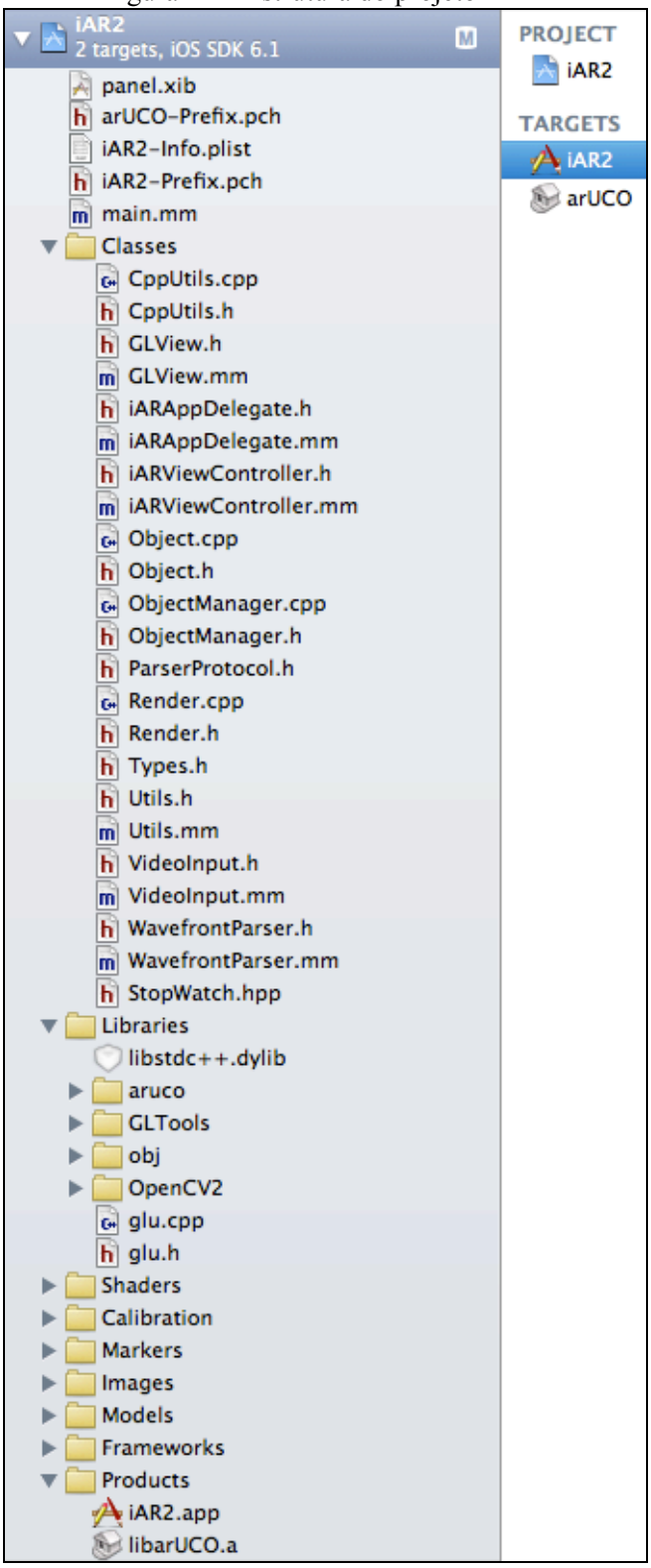
Nesta seção são explorados assuntos pertinentes a utilização deste projeto. Dividida em duas partes, a primeira dedicada a desenvolvedores e a segunda dedicada a usuário e desenvolvedores.

3.3.5.1 Estrutura do projeto e compilação

Alguns aspectos de estrutura do projeto e compilação mudaram desde a versão original. Iniciando pela estrutura, representada na Figura 11, moveu-se os fontes que não são relacionados a uma determinada biblioteca para o grupo `Classes`. A biblioteca ArUco recebeu seu próprio *target* de compilação, isto para ajudar na organização do código e modularizar as configurações de compilação.

Seguindo com a compilação, agora existem dois arquivos de saída, o pacote executável `iAR2.app` e a biblioteca estática `libarUCO.a`. Estes arquivos ficam no grupo `Products` da estrutura do projeto.

Figura 11 – Estrutura do projeto `iAR2`



Outra diferença importante do projeto original, é quanto às *flags* que informam ao compilador para utilizar instruções do módulo NEON, `-mfloat`, `-abi=softfp` e `-mfpu=neon`. Estas *flags* são necessárias somente no *target* `arUCO`, já que não é utilizado código SIMD nos demais fontes. A Figura 12 traz as configurações mais importantes do *target*.

Figura 12 – Configuração do *target* `arUCO`

Architectures	
Additional SDKs	
Architectures	Standard (armv7, armv7s) – $(ARCHS_STANDARD_32_BIT)
Base SDK	Latest iOS (iOS 6.1)
Build Active Architecture Only	**No**
Supported Platforms	iOS
Valid Architectures	armv7 armv7s
Apple LLVM compiler 4.2 – Code Generation	
Optimization Level	**Fast [-O, O1]**
Apple LLVM compiler 4.2 – Language	
Other C Flags	**-mfloat-abi=softfp -mfpu=neon**

Foram realizados vários testes de estabilidade para definir o nível de otimização no compilador. O único nível que não causou problemas de execução foi o `Fast [-O,O1]`, portanto este foi escolhido. O *target* `iAR2` tem sua configuração similar a do anterior, com a diferença de que não são necessárias as *flags* do módulo NEON. A Figura 13 ilustra estas configurações.

Figura 13 – Configuração do *target* `iAR2`

Architectures	
Additional SDKs	
Architectures	Standard (armv7, armv7s) – $(ARCHS_STANDARD_32_BIT)
Base SDK	Latest iOS (iOS 6.1)
Build Active Architecture Only	**No**
Supported Platforms	iOS
Valid Architectures	armv7 armv7s
Apple LLVM compiler 4.2 – Code Generation	
Optimization Level	**Fast [-O, O1]**

Por fim, é necessário criar *links* com as bibliotecas as quais o código depende. Como o *target* `iAR2` gera o executável, ele é quem faz o *link* entre o código fonte e as bibliotecas. Na Figura 14 temos a lista de bibliotecas que são utilizadas durante a compilação.

Figura 14 – Lista de bibliotecas utilizadas na compilação

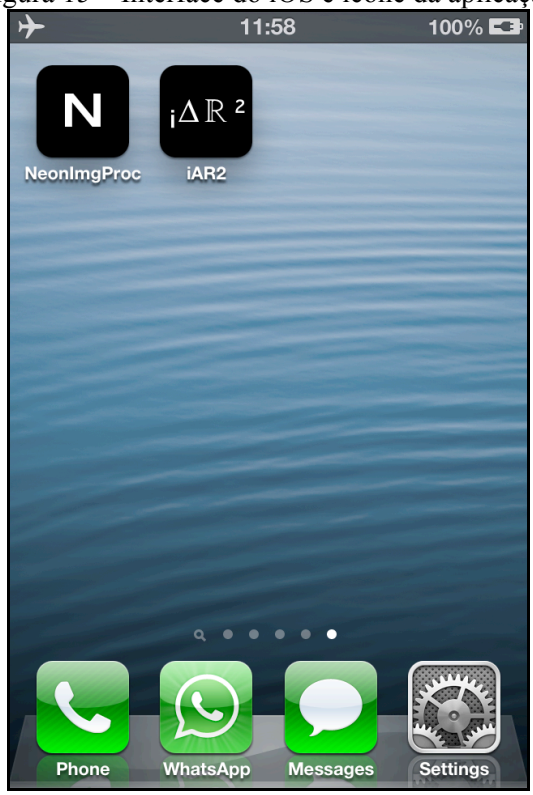
Link Binary With Libraries (12 items)
libopencv_calib3d.a
libopencv_core.a
libopencv_imgproc.a
libarUCO.a

3.3.5.2 Aplicação de testes

Considerando que a compilação executou com sucesso e o aplicativo está armazenado no dispositivo, é possível fazer a execução do mesmo. Para executar o aplicativo, bastar tocar

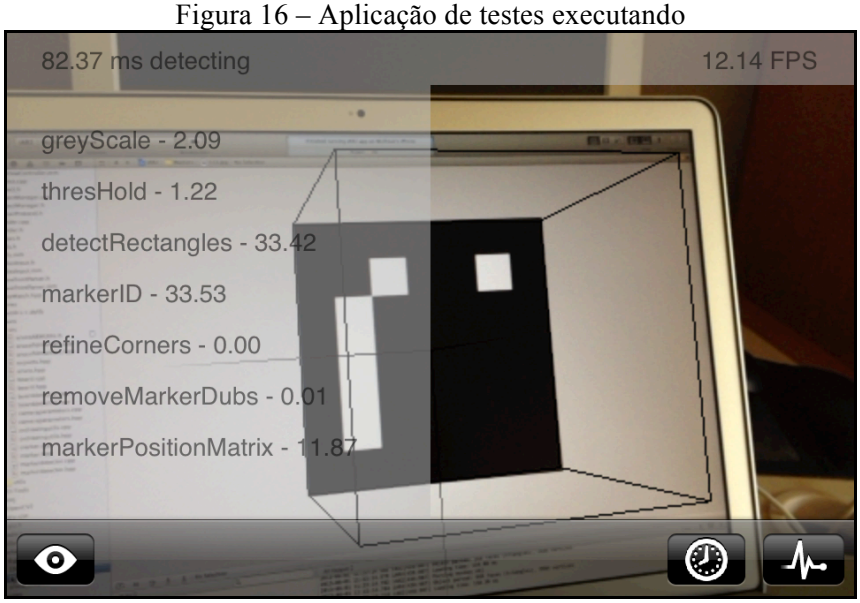
no ícone presente na interface do dispositivo. O ícone tem o nome do projeto, iAR2, escrito em branco. A Figura 15 ilustra a interface do iOS com o ícone da aplicação.

Figura 15 – Interface do iOS e ícone da aplicação



A aplicação inicia já executando o processo de RA, o usuário podendo acionar os detalhes de desempenho com os botões à direita. A Figura 16 traz uma captura de tela onde a aplicação mostra todas as interfaces disponíveis ao usuário.

Figura 16 – Aplicação de testes executando



3.4 RESULTADOS E DISCUSSÃO

Iniciou-se este trabalho com a adaptação do código de Hess (2011b) para as novas versões das tecnologias utilizadas. Estas sendo a versão 6.1 da SDK do iOS, o dispositivo iPhone 4S, a versão 2.4.4 da biblioteca OpenCV e a versão 1.2.4 da biblioteca ArUco. Esta tarefa provou-se trabalhosa, e em certos momentos, complexa. Destaca-se nesta lista a compilação da biblioteca OpenCV, que já havia sido um obstáculo para Hess (2011a, p. 41). A atualização da biblioteca ArUco ocorreu com facilidade, mostrando o compromisso do autor (SALIÑAS, 2012) com a compatibilidade de seu código.

Em seguida, as tarefas de otimização introduziram um novo fator de dificuldade, a codificação em *assembler*. Inicialmente tentou-se codificar diretamente do algoritmo protótipo para o *assembler*, ignorando os passos recomendados por Merel (2011) e Pipenbrinck (2009). Este foi um grave erro pois desconsiderou-se a adaptação de código em série para código paralelo com instruções SIMD. Percebeu-se então que é essencial passar por todas as etapas, para que o código seja preparado para trabalhar com vários valores por iteração, neste caso eram oito.

Uma vez dominados os passos recomendados para a codificação *assembler*, o desenvolvedor passa a entender os recursos utilizados pelo compilador para agilizar o código. Com este domínio, pode-se criar rotinas em C ou C++ já com o foco na implementação *assembler*. Um exemplo são as estruturas de loop, que no *assembler*, são feitas com contadores decrescentes onde no final de cada iteração compara-se o valor com zero. Este modo de codificar dispensa o uso de um registrador para armazenar o valor máximo. O Quadro 8 traz um exemplo de código criado com a conversão em mente. Neste trecho, que faz parte do método de *threshold*, temos duas versões do código. O código a esquerda, escrito em C++, executa exatamente as mesmas operações que o código *assembler* a direita.

Quadro 8 – Comparação entre código C++ e *assembler*

```
1   ...
2   do {
3      int r5 = 4;
4      int* r6 = (int*)indices;
5      uint8x8_t d0 = vmov_n_u8((uint8_t)0);
6      do {
7         int r7 = *r6;
8         uint8_t* r8 = rSrc + r7;
9         uint8x8_t d1 = vld1_u8(r8);
10        vsra_n_u8(d0, d1, 2);
11        r6 += 4;
12        r5--;
13     }while(r5);
14     vst1_u8(rDst, d0); rDst += 8;
15     rSrc += 8;
16     r4--;
17  } while(r4);
18  ...
```

```
...
1:
mov         r5, #4
mov         r6, %[i]
vmov.u8     d0, #0
2:
ldr         r7, [r6]
add         r8, %[src], r7
vld1.8      {d1}, [r8]
vsra.u8     d0, d1, #2
add         r6, #4
subs        r5, r5, #1
bne         2b
vst1.8      {d0}, [%[dst]]!
add         %[src], #8
subs        r4, r4, #1
bne         1b
...
```

As próximas seções descrevem detalhes das mudanças que ocorreram em comparação com a versão original da API. Foram analisados consumo de memória, desempenho e características. Para viabilizar as comparações, a aplicação de testes iAR original foi compilada para o sistema operacional iOS 6.1 no dispositivo iPhone 4S. Desta forma as duas versões executaram nas mesmas condições.

3.4.1 Consumo de memória

O consumo de memória mostrou-se estável ao longo da execução da aplicação. A ferramenta Instruments, do ambiente Xcode, mostrou a aplicação mantendo uma média de 2,54 MB de memória alocada. O Quadro 9 contém a comparação realizada entre a versão original e a atual. Foram avaliadas situações onde as aplicações estavam executando com todas as interfaces habilitadas e detectando um, dois e três marcadores.

Quadro 9 – Consumo de memória

Número de marcadores	Memória (Megabytes)	
	iAR	iAR2
1	2,52	2,53
2	2,53	2,54
3	2,54	2,55

A expectativa inicial era de que a versão atual consumiria menos memória do que a original. Porém, observou-se que a atual consumiu pouco mais, o equivalente a 10,24 KB. Este aumento de consumo deve-se a inclusão de novos componentes na interface. Considera-se também que os trechos de código feitos em *assembler*, reduziram o consumo. Então, pode-se concluir que o aumento no consumo seria maior sem as novas rotinas.

3.4.2 Desempenho

Os testes realizados com a nova versão, mostraram um aumento considerável no desempenho em comparação com a original. O Quadro 10 contém dois comparativos entre as versões original e atual, estas detectando de um a três marcadores. O comparativo a esquerda trabalha com a taxa de quadros por segundo de cada versão, onde quanto maior o valor mais rápida a execução ocorreu. O comparativo a direita mostra o tempo consumido para realizar o processo de detecção em um quadro.

Quadro 10 – Desempenho global da aplicação

Número de marcadores	Quadros por segundo		Tempo de detecção (milissegundos)	
	iAR	iAR2	iAR	iAR2
1	8,50	13,00	126,20	75,15
2	7,40	11,00	140,33	90,50
3	5,91	10,13	170,12	100,53

Notou-se um ganho médio de 57% nas taxas de quadros por segundo. No tempo de detecção o ganho foi de 39%. Estes ganhos em desempenho são o resultado de várias modificações que ocorreram desde a versão original. Dentre elas, cita-se a compilação das bibliotecas OpenCV e ArUco com as opções de otimização do compilador ativadas. A atualização da *pipeline* de RA foi onde ocorreram os maiores ganhos, tanto nas rotinas implementadas em *assembler*, como na atualização da biblioteca ArUco. O Quadro 11 traz novamente um comparativo, este trazendo o tempo de cada etapa do processo de RA durante o reconhecimento de somente um marcador.

Quadro 11 – Desempenho de cada etapa de RA

Etapa de RA	Tempo de execução (milissegundos)	
	iAR	iAR2
Escala de cinza	2,35	2,35
Threshold	40,33	1,30
Detectar retângulos	40,13	33,42
Identificação marcadores	28,93	24,12
Refinar cantos	1,05	0,05
Remover duplicatas	0,05	0,02
Calcular matriz de posição	13,15	11,72

Observando o Quadro 11, nota-se um ganho de desempenho por todo o processo de RA, com exceção da etapa de escalas de cinza. Esta etapa já tinha sido convertida para *assembler* por Hess (2011a, p. 52). A etapa onde ocorreu o maior ganho de tempo foi o método de *threshold*. Com um tempo 31 vezes mais rápido que a versão original, o método codificado com *assembler* e instruções SIMD provou a eficácia destas tecnologias. É

importante lembrar que a versão codificada neste trabalho é simplificada em comparação com a utilizada no trabalho anterior. Na versão anterior, a biblioteca ArUco dependia da função de *threshold* implementada na biblioteca OpenCV. Esta função utiliza mecanismos complexos que garantem reutilização de código. Portanto, na versão atual optou-se por focar na necessidade da *pipeline* de RA.

Para evidenciar o processo de codificação *assembler* com SIMD, é apresentado o Quadro 12. Observa-se a evolução no ganho de tempo através das três versões do algoritmo de *threshold*.

Quadro 12 – Desempenho das três versões do algoritmo de *threshold*

Tipo de codificação	Tempo de execução (milissegundos)
C++	13,35
C++ e SIMD	8,50
Assembler e SIMD	1,30

Analisando o Quadro 12 nota-se que mesmo executando em paralelo, a versão escrita em C++ com chamadas *intrinsics* não melhora significativamente o desempenho. Isto deve-se ao fato de que o compilador continua fazendo acessos desnecessários a memória. Criando código *assembler* diretamente, pode-se utilizar somente registradores, o que é o caso desta implementação. Com isto, economiza-se preciosos 7,2 milissegundos.

3.4.3 Características

As características principais do projeto original foram mantidas, com a adição de suporte a marcadores compostos. O Quadro 13 traz um comparativo entre a versão original, a versão atual e os trabalhos correlatos.

O suporte a marcadores compostos foi introduzido na atualização da biblioteca ArUco. Marcadores compostos são matrizes onde marcadores individuais são posicionados de forma a serem reconhecidos pelo motor de RA como um único marcador. Infelizmente não houve tempo para explorar este novo recurso.

Quadro 13 – Comparação entre trabalhos correlatos

Características / Nome dos Trabalhos	iAR	iAR2	AR Defender	Vuforia
Número máximo de marcadores	1024	1024	1	Ilimitado, recomendado 50
Suporte a marcadores compostos	Não	Sim	Não	Sim
Suporte a marcadores criados pelo usuário	Não	Não	Não	Sim
Plataformas	iOS	iOS	iOS, Android, BadaOS	iOS, Android, Unity 3D
Tipo de integração	Ferramenta	Ferramenta	Aplicação	SDK, API, *Plugin*

4 CONCLUSÕES

O trabalho abordou a otimização de uma ferramenta existente para que esta explorasse melhor os recursos das tecnologias nela envolvidas. Nota-se que após a integração de novas versões das bibliotecas, houve um ganho de desempenho significativo através de toda a aplicação.

A tecnologia onde colocou-se maior parte dos esforços, foi o módulo SIMD dos processadores ARM, o NEON. Este recurso dos processadores provou ter grande potencial em melhoria de desempenho, possibilitando o processamento de até 8 valores simultaneamente. Juntamente com a programação em *assembler*, as instruções SIMD provaram ser a mais poderosa ferramenta, em termo de desempenho, nos dispositivos móveis.

Para tornar possível as compilações executadas neste projeto, tanto da própria aplicação como das bibliotecas envolvidas, foi necessário utilizar conhecimentos adquiridos de forma independente do curso. Estes conhecimentos provaram ser imprescindíveis, pois sem eles este projeto não teria sido concluído.

Apesar da taxa de quadros por segundo ainda não ter atingido o limiar de trinta, já pode-se notar uma melhora significativa no tempo de resposta da aplicação. Nota-se também que a API foi testada em um dispositivo iPhone 4S. Portanto, recomenda-se o uso do projeto em dispositivos equivalentes ou superiores em processamento.

Os recursos do ambiente Xcode foram amplamente utilizados, juntamente com ferramentas como o cliente SourceTree e o serviço de repositórios BitBucket. Todas as ferramentas são gratuitas e o serviço de repositórios também é gratuito para projetos privados com times de até cinco desenvolvedores. A ferramenta Instruments foi essencial tanto no desenvolvimento do código como na descrição dos resultados, pois esta monitora memória e busca vazamentos.

O conjunto tema e técnicas utilizadas provou ser um encontro perfeito de problema e solução. Tendo atingido boa parte dos objetivos, conclui-se ainda que é necessário mais tempo de dedicação exclusiva ao desenvolvimento das rotinas otimizadas.

4.1 EXTENSÕES

Para trabalhos futuros, são sugeridas as seguintes extensões:

a) otimizar ainda mais o motor de detecção, utilizando *threads*;
b) explorar os recursos de marcadores compostos da biblioteca ArUco;
c) implementar o reconhecimento de marcadores criados pelo usuário como, por exemplo, imagens;
d) desenvolver mais a parte de renderização de objetos 3D, adicionando texturização e interações entre este objetos;
e) tornar a API mais independente da plataforma iOS, visando compilar para outras plataformas que utilizem processadores ARM como, por exemplo, Android.

REFERÊNCIAS BIBLIOGRÁFICAS

ACCELEREYES. **Jacket NDK:** mobile gpu computing for accelerated apps. Atlanta, 2011. Disponível em: <http://www.accelereyes.com/products/mobile>. Acesso em: 15 set. 2012.

ARM LTD. **The ARM Cortex-A9 processors**. Cambridge, 2009. Disponível em: <http://www.arm.com/files/pdf/ARMCortexA-9Processors.pdf>. Acesso em: 7 set. 2012.

_____. **NEON**. Cambridge, 2012. Disponível em: <http://www.arm.com/products/processors/technologies/neon.php>. Acesso em: 7 set. 2012.

APPLE INC. **Apple launches iPhone 4S, iOS 5 & iCloud**. Cupertino, 2011. Disponível em: <http://www.apple.com/pr/library/2011/10/04Apple-Launches-iPhone-4S-iOS-5-iCloud.html>. Acesso em: 31 ago. 2012.

_____. **iOS 6:** which software features does my iPhone, iPad, or iPod touch support. Cupertino, 2012. Disponível em: <http://support.apple.com/kb/HT5457>. Acesso em: 26 maio 2013.

AZUMA, Ronald T. A survey of augmented reality. In: PRESENCE: TELEOPERATORS AND VIRTUAL ENVIRONMENTS, 6th, 1997, [S.l.]. **Proceedings...** [S.l.: s.n.], 1997. p. 355-385. Disponível em: <http://www.cs.unc.edu/~azuma/ARpresence.pdf>. Acesso em: 1 set. 2012.

BOGGAN, Sha'Kia; PRESSEL, Daniel M. **GPUs:** an emerging platform for general-purpose computation. Aberdeen, 2007. 42 f. Disponível em: <http://www.arl.army.mil/arlreports/2007/ARL-SR-154.pdf>. Acesso em: 9 set. 2012.

CAMARGO, Állan M. **Repositório iAR2**. Blumenau, 2013. Disponível em: <http://bitbucket.org/amcamargo/iar2>. Acesso em: 23 maio 2013.

FLYNN, Michael J. **Some computer organizations and their effectiveness**. [S.l.], 1972. Disponível em: <http://regal.csep.umflint.edu/~swturner/Classes/csc578/Papers/Flynn72.pdf>. Acesso em: 13 jul. 2013.

HESS, Jonathan. **Explorando modelos virtuais 3D com realidade aumentada no SDK do Iphone**. 2011a. 76 f. Trabalho de Conclusão de Curso (Bacharelado em Ciências da Computação) - Centro de Ciências Exatas e Naturais, Universidade Regional de Blumenau, Blumenau. Disponível em: <http://www.bc.furb.br/docs/MO/2011/348549_1_1.pdf>. Acesso em: 1 set. 2012.

_____. **Repositório iAR**. Blumenau, 2011b. Disponível em: <http://xp-dev.com/svn/iAR/trunk/>. Acesso em: 1 mar. 2013.

INT13. **AR Defender**. [Paris], 2010. Disponível em: <http://www.ardefender.com/>. Acesso em: 4 set. 2012.

INTEL CORPORATION. **Schema validation with Intel® streaming SIMD extensions 4**. Santa Clara, 2008. Disponível em: <http://software.intel.com/en-us/articles/schema-validation-with-intel-streaming-simd-extensions-4-intel-sse4>. Acesso em: 9 maio 2013.

KEFALLONITIS, Fivos. **Name mangling demystified**. [S.l.], 2007. Disponível em: <http://www.int0x80.gr/papers/name_mangling.pdf>. Acesso em: 26 maio 2013.

KHRONOS GROUP. **OpenGL ES 2.0**. San Francisco, 2007. Disponível em: <http://www.khronos.org/news/press/2007/03>. Acesso em: 9 set. 2012.

KIPP, Harald. **ARM GCC inline assembler cookbook**. [S.l.], 2012. Disponível em: <http://www.ethernut.de/en/documents/arm-inline-asm.html>. Acesso em: 10 abr. 2013.

KITWARE. **CMake:** the cross-platform, open-source build system. [S.l.], 2013. Disponível em: <http://www.cmake.org>. Acesso em: 12 mar. 2013.

LEUPERS, Rainer. **Code selection for media processors with SIMD Instructions**. Dortmund, 2000. Disponível em: <http://www.date-conference.com/proceedings/PAPERS/2000/DATE00/PDFFILES/01A_1.PDF>. Acesso em: 15 set. 2012.

MEREL, Steve. **NEON optimizations for iOS and Android**. [S.l.], 2011. Disponível em: <http://www.crickettechnology.com/blog/?p=691>. Acesso em: 29 mar. 2013.

OPENCV WIKI. **Welcome**. [S.l.], 2013. Disponível em: <http://opencv.willowgarage.com/wiki/>. Acesso em: 12 mar. 2013.

PIPENBRINCK, Nils. **ARM NEON optimization:** an example. [S.l.], 2009. Disponível em: <http://hilbert-space.de/?p=22>. Acesso em: 30 mar. 2013.

QUALCOMM INC. **Augmented reality (Vuforia)**. [S.l.], 2012a. Disponível em: <https://developer.qualcomm.com/mobile-development/mobile-technologies/augmented-reality>. Acesso em: 16 set. 2012.

_____. **Run the sample application**. [S.l.], 2012b. Disponível em: <https://ar.qualcomm.at/sdk/ios>. Acesso em: 16 set. 2012.

SALIÑAS, Rafael T. **ArUco:** augmented reality library from the University of Cordoba. Córdoba, 2012. Disponível em: <http://sourceforge.net/projects/aruco/files/1.2.4/>. Acesso em: 19 mar. 2013.

WAGNER, Daniel; SCHMALSTIEG, Dieter. **First steps towards handheld augmented reality**. [S.l.], 2003. Disponível em: <http://www.icg.tu-graz.ac.at/Members/daniel/Publications/HandheldAR_ISWC03final.pdf>. Acesso em: 30 ago. 2012.

APÊNDICE A – Detalhamento do método `neonAdaptiveThreshold`

Neste apêndice são apresentadas as versões criadas do loop principal do método `neonAdaptiveThreshold`. O Quadro 14 apresenta a versão deste código com o loop original, sem otimizações.

Quadro 14 – Versão original do loop de *threshold*

```
    ...
1   uchar tab[768];
2
3   for( i = 0; i < 768; i++ )
4   {
5       tab[i] = (uchar)(i - 255 <= -idelta ? imaxval : 0);
6   }

    ...

7   for( i = 0; i < size.height; i++ )
8   {
9       const uchar* sdata = src.data + src.step*i;
10      const uchar* mdata = mean.data + mean.step*i;
11      uchar* ddata = dst.data + dst.step*i;
12
13      for( j = 0; j < size.width; j++ )
14          ddata[j] = tab[sdata[j] - mdata[j] + 255];
15  }
    ...
```

No código, o trecho entre as linhas `1` e `6` é a preparação da tabela `tab` de onde são buscados os valores no loop principal. Esta tabela é preenchida com zero ou o valor máximo, preto e branco respectivamente. No loop principal `(7)` são acessados valores da imagem original `(9)` e da imagem filtrada `(10)`. A diferença desses valores é utilizada para definir qual valor da tabela será utilizado para designar ao pixel `(14)`.

No Quadro 15 temos a primeira versão otimizada com SIMD. Para utilizar as instruções SIMD sem trabalhar com código *assembler*, o desenvolvedor deve chamar funções do tipo *intrinsics*.

Quadro 15 – Versão otimizada com *intrinsics*

```
   ...
1  uint8_t* sdata = src.data;
2  uint8_t* mdata = mean.data;
3  uint8_t* ddata = dst.data;
4  const uint8_t* sEnd = sdata + (size.width * size.height);
5
6  const int8x8_t deltaN = vdup_n_s8(-delta);
7  const uint8x8_t imaxvalN = vdup_n_u8(imaxval);
8  const uint8x8_t zeroN = vdup_n_u8(0);
9
10 while(sdata < sEnd)
11 {
12     const uint8x8_t sdataN = vld1_u8(sdata);
13     const uint8x8_t mdataN = vld1_u8(mdata);
14     const int8x8_t op1 = vsub_s8(sdataN, mdataN);
15     uint8x8_t mask = vcle_s8(op1, deltaN);
16     vbsl_u8(mask, imaxvalN, zeroN);
17
18     vst1_u8(ddata, mask);
19
20     sdata += 8;
21     mdata += 8;
22     ddata += 8;
23 }
   ...
```

A principal diferença da versão anterior é que nesta são processados oito valores a cada iteração do loop. As linhas de `6` a `8` inicializam registradores vetoriais com as constantes envolvidas no loop. Nota-se que nesta versão não existe mais a tabela de referência para busca dos valores. A tabela foi removida pois não é mais necessária, o valor é definido durante o loop. Na linha `14` é realizada a diferença entre os valores das imagens original e filtrada. A linha `15` cria uma máscara de bytes comparando o resultado da linha anterior com o delta. O Quadro 16 demonstra abstratamente a execução da instrução `cle` (neste caso `vcle_s8` da linha `15`), que significa *compare less or equal* (comparar menor ou igual).

Quadro 16 – Execução da instrução `cle`

```
op1     | 7 5 9 2  8 1 2 3
deltaN  | 7 7 7 7  7 7 7 7
--------+---------------------
mask    | 1 1 0 1  0 1 1 1
```

Baseando-se na máscara gerada, a linha `16` designa zero ou o valor máximo para cada pixel no registrador vetorial. A linha `18` faz o armazenamento em memória dos oito pixels processados na iteração atual. Por fim, as linhas `20` a `22` incrementam os ponteiros para a próxima iteração.

A última versão do código, escrita em *assembler*, está no Quadro 17.

Quadro 17 – Versão codificada em *assembler*

```
   ...
1  __asm__ __volatile__
2  ("lsr       %[sz], %[sz], #3    \n"
3   "vdup.s8   d0, %[delta]        \n"
4   "vdup.u8   d1, %[maxval]       \n"
5   "vmov.u8   d2, #0              \n"
6   "1:                            \n"
7   "vld1.8    {d3}, [%[src]]!     \n"
8   "vld1.8    {d4}, [%[m]]!       \n"
9   "vsub.s8   d5, d3, d4          \n"
10  "vcle.s8   d6, d5, d0          \n"
11  "vbsl.u8   d6, d1, d2          \n"
12  "vst1.8    {d6}, [%[dst]]!     \n"
13  "subs      %[sz], %[sz], #1    \n"
14  "bne       1b                  \n"
15  : [dst] "+r" (dst.data)
16  : [src] "r" (src.data), [m] "r" (mean.data), [sz] "r"
    (size.width*size.height),[maxval] "r" (imaxval),[delta] "r" (-delta)
17  : "memory", "cc");
   ...
```

Com a codificação *assembler* reduz-se significativamente o acesso a memória, já que são utilizados somente registradores para as operações. Nesta versão não foi modificada a forma como são processados os pixels, esta utilizando a mesma sequência de instruções da primeira versão otimizada (com SIMD, Quadro 15). Na linha `13` é feito o decremento do contador e atualizadas as variáveis de condição do processador. Na linha `14` ocorre o loop caso a última comparação não tenha sido igual a zero. As linhas `15` a `17` declaram registradores de entrada e saída e também quais recursos serão utilizados no bloco.

ANEXO A – Casos de uso da aplicação de testes

Neste anexo encontram-se descrições detalhadas dos casos de uso UC01 a UC05 presentes no diagrama da Figura 5.

O caso de uso UC01 descreve a funcionalidade que permite ao `Usuário` visualizar a entrada de vídeo da câmera. Maiores detalhes estão no Quadro 18.

Quadro 18 – Caso de uso UC01 em detalhes

UC01 – Visualizar imagens da entrada de vídeo	
Descrição	Permite ao `Usuário` visualizar as imagens capturadas pela câmera digital do dispositivo em tempo real.
Pré-Condição	O dispositivo e o SDK devem oferecer suporte ao uso da câmera digital.
Cenário principal	1. O `Usuário` instância um objeto `iARViewController`; 2. O objeto controlador cria uma instância da classe `VideoInput`; 3. A instância de `VideoInput` executa as operações necessárias para a captura da entrada de vídeo; 4. O `Usuário` acopla o objeto criado à janela do dispositivo.
Exceção	Na etapa 3, a entrada de vídeo pode ser recusada quando o dispositivo e o SDK não oferecerem suporte à câmera digital.
Pós-condição	O `Usuário` passa a ser capaz de visualizar imagens da entrada de vídeo na tela do dispositivo em tempo real.

Fonte: Hess (2011a, p. 31).

O caso de uso UC02 descreve a relação entre o `Usuário` e a funcionalidade que renderiza os objetos 3D criando o efeito de RA. Maiores detalhes estão no Quadro 19.

Quadro 19 – Caso de uso UC02 em detalhes

UC02 – Visualizar objetos 3D na tela	
Descrição	Permite ao `Usuário` visualizar objetos 3D renderizados pelo OpenGL ES sobrepondo as imagens da entrada de vídeo em tempo real.
Pré-condição	O dispositivo e o SDK devem oferecer suporte ao OpenGL ES 2.0.
Cenário principal	1. O `Usuário` instância um objeto `iARViewController`; 2. O objeto controlador cria uma instância da classe `GLView`; 3. A instância de `GLView` executa as operações necessárias para a renderização de objetos 3D; 4. O `Usuário` acopla o objeto criado à janela do dispositivo.
Exceção	Na etapa 3, o objeto `GLView` pode identificar que o dispositivo e o SDK não suportam a versão do OpenGL indicada, deixando de desenhar quaisquer coisas.
Pós-condição	O `Usuário` passa a ser capaz de visualizar objetos 3D sobrepondo imagens da câmera digital na tela do dispositivo em tempo real.

Fonte: Hess (2011a, p. 32).

O caso de uso UC03 descreve a opção disponibilizada ao `Usuário` que permite os objetos 3D serem rotacionados através de toques na tela. Detalhes são apresentados no Quadro 20.

Quadro 20 – Caso de uso UC03 em detalhes

UC03 – Explorar objetos 3D com toques	
Descrição	Permite ao `Usuário` interagir diretamente com os objetos 3D visíveis na tela do dispositivo. Nesta implementação somente a rotação dos objetos será disponibilizada.
Pré-condição	Devem haver objetos 3D visíveis na tela do dispositivo.
Cenário principal	1. O `Usuário` inicializa o motor de RA criando um objeto `iARViewController` e acoplando-o à janela do dispositivo; 2. O `Usuário` apresenta marcações à entrada de vídeo; 3. As rotinas do motor de RA reconhecem as marcações e renderizam objetos 3D na tela do dispositivo; 4. O `Usuário` inicia o toque na tela do dispositivo; 5. O `Usuário` arrasta o dedo para outra posição da tela; 6. O objeto controlador altera a rotação dos objeto 3D; 7. O motor de renderização desenha os objetos na nova rotação recebida pelo objeto controlador; 8. O `Usuário` solta o dedo da tela do dispositivo.
Exceção	Na etapa 5, o objeto controlador pode identificar que nenhum objeto está presente na tela, e toda a operação é ignorada.
Pós-condição	O `Usuário` passa a ser capaz de explorar os objetos 3D visíveis através de toques na tela do dispositivo.

Fonte: Hess (2011a, p. 33).

O caso de uso UC04 descreve a opção disponível ao `Usuário` que permite alternar entre os modos de visualização dos gráficos 3D. A descrição completa está presente no Quadro 21.

Quadro 21 – Caso de uso UC04 em detalhes

UC04 – Selecionar entre modos de visualização	
Descrição	Permite ao `Usuário` selecionar entre dois possíveis modos de visualização dos gráficos 3D. O modo padrão é a visualização de uma *bounding box* (caixa delimitadora) aramada junto dos eixos de orientação do objeto, ambos exatamente dentro dos perímetros ocupados pelas marcações encontradas. A segunda opção é a visualização dos objetos 3D típicos, desenhados com polígonos pelo OpenGL.
Cenário principal	1. O `Usuário` carrega a aplicação de RA; 2. O motor de RA renderiza os objetos 3D no modo padrão; 3. O `Usuário` toca no botão de seleção dos modos de visualização; 4. A aplicação inverte o valor da *flag* que armazena o tipo de renderização adotada; 5. A classe `Render` desenha o objeto 3D de acordo com o tipo de visualização.
Pós-condição	O `Usuário` passa a ser capaz de selecionar entre os modos de visualização dos objetos 3D renderizados através da interface da aplicação.

Fonte: Hess (2011a, p. 33).

O caso de uso UC05 descreve o monitoramento de desempenho da ferramenta, disponível ao `Usuário` através de indicadores visuais. Maiores detalhes estão no Quadro 22.

Quadro 22 – Caso de uso UC05 em detalhes

UC05 – Monitorar o desempenho da ferramenta	
Descrição	Permite ao `Usuário` monitorar dois diferentes tipos de medição do desempenho da ferramenta. A primeira medição é relativa ao tempo de execução do algoritmo de detecção das marcações, e a segunda, a quantidade de quadros por segundo que o dispositivo consegue gerar dentro desta taxa de processamento.
Pré-condição	O motor de RA deve estar executando o algoritmo de detecção de marcações.
Cenário principal	1. O `Usuário` carrega a aplicação de RA; 2. O motor de RA executa todas as rotinas padrão; 3. O `Usuário` toca no botão de monitoração do desempenho da aplicação; 4. A aplicação inverte o valor da *flag* que armazena o status do monitoramento do desempenho da aplicação; 5. A interface mostra a barra com informações das medições no topo da tela ou a esconde de acordo com o status da variável.
Exceção	Na etapa 5, a interface pode deixar de gerar as informações quando o motor de RA não executar o algoritmo de detecção.
Pós-condição	O `Usuário` passa a ser capaz de habilitar a monitoração do desempenho da ferramenta através da interface da aplicação.

Fonte: Hess (2011a, p. 34).

ANEXO B – Descrições das classes

Neste anexo estão as descrições das classes mantidas do diagrama original de Hess (2011a, p. 35). Estas classes estão representadas no diagrama da Figura 6.

As classes `iARAppDelegate` e `iARViewController`, responsáveis por iniciar a aplicação e o motor de RA, são descritas nos Quadros 23 e 24.

Quadro 23 – Descrição da classe `iARAppDelegate`

A classe `iARAppDelegate` é meramente uma classe auxiliar que é chamada pelo método main do SDK do iPhone. Na aplicação de testes, tem finalidades quanto ao carregamento do sistema, preparando todo o contexto do motor de RA para o funcionamento.

Fonte: Hess (2011a, p. 36).

Quadro 24 – Descrição da classe `iARViewController`

A classe `iARViewController` centraliza todas as operações da ferramenta. Ela tem a responsabilidade de inicializar e despachar todas as atividades do motor de RA, além de estender as propriedades de um componente do SDK do iPhone, o `UIViewController`. Quando estende as propriedades deste controlador, pode ser facilmente vinculado ao contexto de visualização do componente `UIWindow`, que por sua vez é a janela principal das aplicações no iOS. Suas definições também a obrigam a receber e processar notificações do dispositivo quando são detectadas mudanças na orientação, e da classe `VideoInput`, quando um novo *buffer* de imagem está pronto para passar pelo algoritmo de detecção de marcações. Referente ao motor de detecção de marcações, três importantes atributos são definidos nesta classe, são eles `cameraParams`, `detector` e `markers`, todos derivados da biblioteca *Augmented reality University of cordoba* (ArUco), base do projeto. Esta classe disponibiliza ainda o método `customize`, que pode ser reescrito para adição de novas funcionalidades à ferramenta, assim como acontece com a interface da aplicação de testes, que é na verdade uma alteração da implementação padrão.

Fonte: Hess (2011a, p. 36).

As classes `VideoInput`, `GLView` e `Render`, as quais são responsáveis pela parte gráfica da aplicação, são descritas nos Quadros 25, 26 e 27.

Quadro 25 – Descrição da classe `VideoInput`

A classe `VideoInput` opera com funcionalidades oferecidas pelo SDK do iPhone para a recepção de pacotes provenientes da câmera digital do dispositivo. Sua obrigação é aguardar o recebimento de *buffers* de imagens pelo hardware, transformá-los em um formato aceitável para o algoritmo de detecção e notificar a classe `iARViewController` de que uma nova imagem está pronta para ser utilizada para tal finalidade.

Detalhe é que a classe deve implementar a interface que delega funções de captura de vídeo do SDK, a `AVCaptureVideoDataOutputSampleBufferDelegate`. Só a partir daí, o sistema operacional passa a enviar as imagens capturadas ao motor de RA.

Fonte: Hess (2011a, p. 36).

Quadro 26 – Descrição da classe `GLView`

A classe `GLView` é uma extensão da classe `UIView`, um componente de visão do SDK do iPhone. Ela funciona como interface para o motor renderizador repassar imagens 3D à tela do dispositivo. Também é responsável por encaminhar a lista de marcações encontradas pelo algoritmo de detecção à classe `Render`.

Fonte: Hess (2011a, p. 37).

Quadro 27 – Descrição da classe `Render`

A classe `Render` é o motor responsável por desenhar gráficos 3D. Sua obrigação é manter uma lista contendo as identificações das marcações que estão visíveis no momento. Esta lista é atualizada pelo controlador a cada vez que um novo *buffer* de imagem é processado. O método `Draw` responde desenhando objetos 3D que vinculam-se a estas identificações, sempre em ciclo contínuo.

Sua principal qualidade é ser uma classe assíncrona em relação a todas as outras que consistem o motor de RA. Tal característica consiste em continuamente executar o método `Draw`, mesmo quando não há marcações visíveis. Desta forma, é possível conferir ao motor de RA um maior realismo, mas sem interromper o fluxo normal do algoritmo de detecção de marcações.

Fonte: Hess (2011a, p. 37).

As classes `Object` e `ObjectManager`, responsáveis por armazenar e gerenciar os objetos 3D utilizados na aplicação, são descritas nos Quadros 28 e 29.

Quadro 28 – Descrição da classe `Object`

A classe `Object` incorpora todas as propriedades de um objeto `GLBatch` da biblioteca OpenGL *Tools* (GLTools), um armazenador de dados sobre definições geométricas 3D. Na maior parte das vezes sua utilidade é servir de modelo para todas as outras classes da ferramenta, compatibilizando-as no modo como trabalham suas rotinas.

Fonte: Hess (2011a, p. 37).

Quadro 29 – Descrição da classe `ObjectManager`

A classe `ObjectManager` é um *singleton* (classe que possui uma única instância) responsável por organizar os objetos 3D mantidos pelo sistema. Seu principal método é o `objectWithID`, que recebe a identificação de uma marcação e retorna um ponteiro para um objeto `Object` vinculado à esta identificação, podendo ser diretamente renderizado pela classe Render.

Está diretamente associada à classe `iARViewController`, que inicializa-a, e com sua ajuda, cria objetos 3D de acordo com as necessidades da aplicação.

Fonte: Hess (2011a, p. 38).

As classes utilitárias `RuntimeInfo` e `Flags` são descritas no Quadro 30.

Quadro 30 – Descrição das classes `RuntimeInfo` e `Flags`

As classes `RuntimeInfo` e `Flags` são estruturas definidas em C++ que atuam em todo o sistema como atributos globais do motor de RA. Estes atributos são inicializados pelas classes principais e repassados às classes secundárias.

Ambas tem a finalidade de armazenar parâmetros de inicialização utilizados pela maioria das classes em um único espaço de memória, facilitando o acesso à consultas e alterações que ocorrem em tempo de execução, como é o caso, por exemplo, das *flags* da aplicação de testes.

Fonte: Hess (2011a, p. 38).

As classes `ParserProtocol` e `WavefrontParser`, responsáveis por ler os arquivos contendo os vetores dos objetos 3D, são descritas no Quadro 31.

Quadro 31 – Descrição das classes `ParserProtocol` e `WavefrontParser`

As classes `ParserProtocol` e `WavefrontParser` são motores analisadores de arquivos que definem geometrias de objetos 3D. `ParserProtocol` é na verdade uma interface que define métodos obrigatórios para classes que a implementam, como é o caso de `WavefrontParser`.

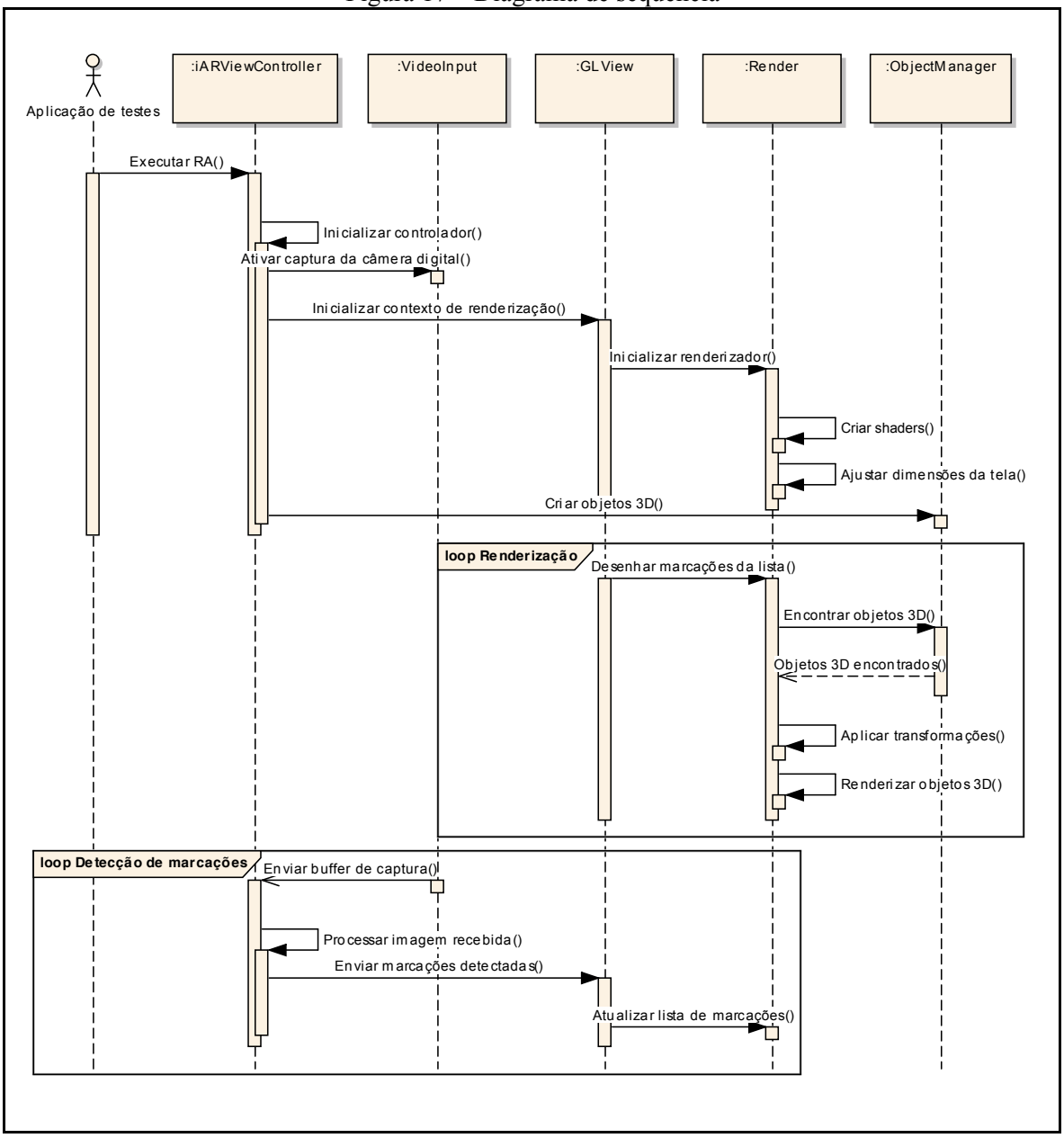
A classe `WavefrontParser` é uma especialização destinada à análise de arquivos com a extensão obj. Ela está diretamente associada à classe `iARViewController`, que a utiliza como meio de criar novos objetos 3D, deixando-os à disposição da classe `Render`.

Fonte: Hess (2011a, p. 38).

ANEXO C – Diagrama de sequência

Neste anexo encontra-se o diagrama de sequência criado por Hess (2011a, p. 39) representando a visão global da aplicação. A Figura 17 contém este diagrama.

Figura 17 – Diagrama de sequência



Fonte: Hess (2011a, p. 39).